



Qualcomm Technologies, Inc.

eAI Integration in CAPI Modules for AudioReach

80-VN500-31 Rev. AA

November 17, 2021

For additional information or to submit technical questions, go to: <https://createpoint.qti.qualcomm.com>

Confidential – Qualcomm Technologies, Inc. and/or its affiliated companies – May Contain Trade Secrets

NO PUBLIC DISCLOSURE PERMITTED: Please report postings of this document on public servers or websites to: DocCtrlAgent@qualcomm.com.

Confidential Distribution: Use or distribution of this item, in whole or in part, is prohibited except as expressly permitted by written agreement(s) and/or terms with Qualcomm Incorporated and/or its subsidiaries.

Not to be used, copied, reproduced, or modified in whole or in part, nor its contents revealed in any manner to others without the express written permission of Qualcomm Technologies, Inc.

All Qualcomm products mentioned herein are products of Qualcomm Technologies, Inc. and/or its subsidiaries.

Qualcomm is a trademark or registered trademark of Qualcomm Incorporated. Other product and brand names may be trademarks or registered trademarks of their respective owners.

This technical data may be subject to U.S. and international export, re-export, or transfer ("export") laws. Diversion contrary to U.S. and international law is strictly prohibited.

Qualcomm Technologies, Inc.
5775 Morehouse Drive
San Diego, CA 92121
U.S.A.

Revision history

Revision	Date	Description
AA	June 2021	Initial release
AB	November 2021	Editorial updates. No technical content was changed.

Qualcomm
Confidential - May Contain Trade Secrets
2026-05-12 06:36:25 GMT
hyungchul.won@lge.com

Contents

1 Introduction.....	5
1.1 Purpose	5
1.2 Conventions	5
1.3 Technical assistance.....	5
2 eAI software architecture.....	6
2.1 Module	7
2.2 eAI runtime	7
2.3 eAI Operator Interface.....	7
2.4 eNPU HAL/Helper library	7
2.5 eAI Math library.....	7
2.6 eNPU hardware driver.....	8
3 eNPU 2.0 overview.....	9
4 API descriptions and call sequences	11
4.1 eAI APIs.....	11
4.2 eAI Utility wrapper APIs	11
4.2.1 Data structures and enumerations	11
4.2.2 API definitions.....	15
4.2.3 Virtual function definitions	16
4.3 eAI Utility call flow	19
4.3.1 Initialization.....	20
4.3.2 IO buffer registration	22
4.3.3 Execute.....	23
4.3.4 De-initialization	23
5 Example.....	25
5.1 Building and running a CAPI-based eAI example detection module on-target	25
5.1.1 Build a CAPI module in the ADSP image	25
5.1.2 Build the SoundTriggerTestApp from the LA build	27
5.1.3 Update resourcemanager.xml file with custom VA config.....	27
5.1.4 Update ACDB files to include eAI-based CAPI module	28
5.1.5 Run end-to-end use case on target.....	34
5.2 CAPI example overview	36
5.2.1 Data structures and enumerations	36
5.2.2 Initialization	38
5.2.3 Execute.....	55

5.2.4 Deinitialization.....	57
6 Power measurements with eNPU enabled/disabled.....	63
6.1 Power measurement with eNPU hardware enabled	63
6.2 Power measurement with eNPU hardware disabled.....	64
A References.....	65
A.1 Related documents.....	65
A.2 Acronyms and terms.....	65

Qualcomm
Confidential - May Contain Trade Secrets
2026-05-12 06:36:25 GMT
hyungchul.won@lge.com

1 Introduction

1.1 Purpose

This document describes how to integrate eAI in CAPI modules in the AudioReach™ framework. It provides guidance on how to prepare an eAI model, invoke eAI runtime APIs in CAPI modules, and run eAI inference in any CAPI module. It also provides details of APIs, best practices, and some example usage.

eAI runtime supports multiple platforms but this document is focused on how to integrate eAI runtime on low power audio subsystem (LPASS) QDSP and LPASS QDSP + eNPU2.0 (AI accelerator) hardware specifically.

1.2 Conventions

Function declarations, function names, type declarations, attributes, and code samples appear in a different font, for example, `cp armcc armcpp`.

Code variables appear in angle brackets, for example, `<number>`.

Commands to be entered appear in a different font, for example, `copy a:*. * b:.`

Button and key names appear in bold font, for example, click **Save** or press **Enter**.

Shading indicates content that has been added or changed in this revision of the document.

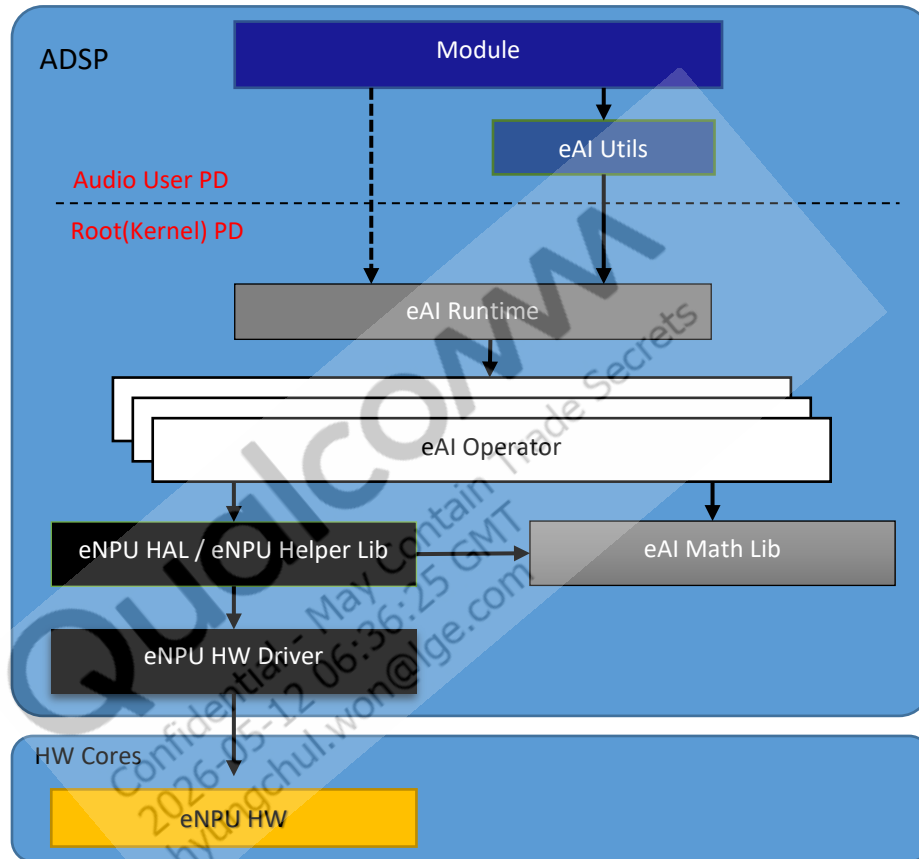
1.3 Technical assistance

For assistance or clarification on information in this document, submit a case to Qualcomm Technologies, Inc. (QTI) at <https://createpoint.qti.qualcomm.com/>.

If you do not have access to the CDMATech Support website, register for access or send email to support.cdmatech@qti.qualcomm.com.

2 eAI software architecture

The following diagram explains the software architecture of the eAI framework from an ADSP perspective.



As shown in the diagram:

- The CAPI module/algorithm calls eAI utils APIs to load the model and do inference with received audio data.

Alternatively, the module/algorithm can choose to call eAI runtime APIs directly from the module/algorithm by managing multiple eAI instances directly inside the module/algorithm (if any)

- The eAI runtime initializes the eAI model and loads the operator library according to the eAI model. Once it receives input data for inference, it executes this data through these operator libraries and passes the result to the CAPI module/algorithm library.
- Based on the configuration received from the CAPI module/algorithm, and based on the operator capabilities, the eAI operator either performs operations on the eNPU hardware and/or the eAI Math library

2.1 Module

A module is CAPI-wrapped ADSP algorithm module. A CAPI module or its underlying algorithm library loads the eAI model in the eAI runtime via exposed eAI utils APIs. The CAPI module also controls whether or not it wants to enable eNPU hardware. It provides real-time input to the eAI runtime for inference and collect the results from the eAI runtime.

2.2 eAI runtime

The eAI runtime or eAI core loads the eAI model and parses the information, such as the number of layers, quantization information, weights and bias information, etc. It loads the eAI operator libraries according to the model requirement, initializes them in the required order, and collects memory requirements from all layers. The eAI runtime also invokes the eNPU if the CAPI module's eNPU usage flag is enabled. Once initialized, the module sends the input data to the eAI runtime. The eAI runtime does inference with the help of eAI operator interface libraries and provides the output back to the module.

2.3 eAI Operator Interface

The eAI operator interface implements various operator layers required for AI/ML processing. The eAI runtime parses the model and loads operators based on the model and chains them together for inference processing.

Some operators have both eNPU and optimized ADSP scalar core-based (eAI Math library) implementation, while some operators have only optimized ADSP scalar core-based (eAI Math library) implementation. Based on the configuration and/or availability, the eAI runtime chooses whether to offload any operator to the eNPU and/or eAI Math library (which is optimized ADSP scalar implementation).

2.4 eNPU HAL/Helper library

This layer helps eAI operator libraries to offload computations to the eNPU. The eNPU HAL and Helper library handle eNPU-related initialization, configuration, and execution. With the help of this library, operators can interact with the eNPU driver and eNPU.

2.5 eAI Math library

eAI Math libraries are optimized scalar implementations of each layer's libraries.

This library either can be used to replace eNPU offloading or do residual processing of eNPU offloading. The module/algorithm developer can choose whether or not to offload computations to the eNPU hardware. If the module/algorithm developer chooses to disable eNPU hardware offloading, all computations are done via these math libraries (which are optimized implementation on ADSP scalar core).

2.6 eNPU hardware driver

The eNPU hardware driver configures the hardware and creates hardware jobs for each client. It handles low level register programming, hardware clock voting, prioritization of jobs, and scheduling of jobs.

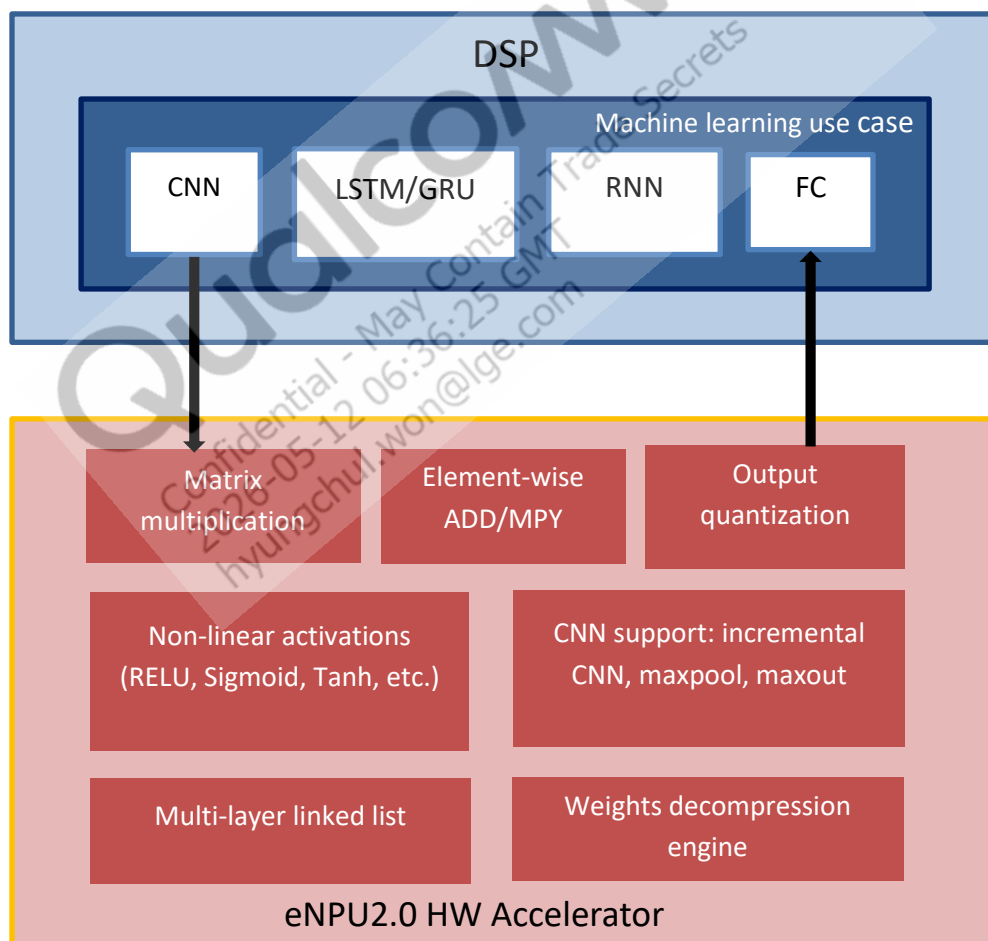
The eAI runtime accumulates eNPU clock and bandwidth requirements from all layers and initializes the hardware using the eNPU hardware driver with the required clock and bandwidth requirements. eAI layers will submit job requests to the eNPU hardware driver during inference.

Qualcomm
Confidential - May Contain Trade Secrets
2026-05-12 06:36:25 GMT
hyungchul.won@lge.com

3 eNPU 2.0 overview

The Qualcomm® Hexagon™ DSP supports vectorized SIMD instructions that are suitable for executing deep neural networks and matrix multiplication code. However, for large networks, and to further lower power consumption, QTI has introduced an accelerator, the eNPU 2.0, that works closely with the DSP to help offload machine learning use cases such as CNN, LSTM/GRU, RNN, and FC with much lower power than QDSP6.

The following figure shows the features supported by eNPU2.0.



eNPU2.0 is a true embedded machine learning accelerator with the following features:

- Dynamic block normalized compression (DBNC) of weights, providing savings in model size. Weights are compressed offline and decompressed by the eNPU on-the-fly using the weight decompression engine to reduce bandwidth.
- Multi-layer linked lists support chaining of operations to reduce interruptions to the DSP
- Element-wise (ADD/MPY) operations help offload complex layers such as LSTM/GRU
- Supports direct convolution, pooling (maxpool and average pool), and maxout operations
- Supports linear and non-linear activation functions such as ReLU, Sigmoid, and Tanh using cubic spline approximation

The eNPU2.0 hardware can operate in Island mode as well as Non-Island mode.

- When operating in Island mode, all input/output data and weights are accessed by the eNPU from the DSP's TCM memory. The eNPU has the ability to access it directly.
- When operating in Non-Island mode, input/output data and weights can be stored in external DDR, and the eNPU can access the data from DDR

Qualcomm
Confidential - May Contain Trade Secrets
2026-05-12 06:36:25 GMT
hyungchul.won@lge.com

4 API descriptions and call sequences

4.1 eAI APIs

Refer to Hexagon SDK documentation (`HEXAGON_SDK_PATH>\addons\eAI\`
`eai_runtime\docs\index.html`) for eAI API definitions and descriptions. The Hexagon
SDK is available for download in Qualcomm Package Manager.

4.2 eAI Utility wrapper APIs

The eAI Utility wrapper is developed on top of the eAI APIs. This utility wrapper provides the following benefits:

- Acts as an adapter by abstracting eAI API dependencies from the SPF framework (CAPI) command/call sequence
- Handles MLA driver dependencies such as machine learning acceleration (MLA) core power up/power down to reduce latency
- Segregates the responsibility between CAPI and CSIM by supporting configuration by CAPI and CSIM (algorithm). For example, framework-specific information such as MLA and LPI information is configured by CAPI whereas eAI instance creation and execution is handled by the algorithm.
- Capability to share scratch memory for multiple instances of eAI from the same algorithm. For example, the same model being run on multiple channels sequentially using multiple eAI instances can share scratch memory.
- Helps CAPI/CSIM offload eAI buffer registration responsibility

4.2.1 Data structures and enumerations

4.2.1.1 eai_util_prop

This enum contains all possible property values used to configure eAI Utility instances and/or get required information from eAI Utility instances.

Property	Description
EAI_UTIL_PROP_BASE_ID	Base property ID. Value = 0x10000000
EAI_UTIL_PROP_REUSE_SCRATCH_MEM	Used to configure utility to re-use scratch memory across all eAI instances hosted by it. This property is set and supported only at eAI Utility init. ■ Default: Enabled ■ struct: eai_util_reuse_scratch_t

Property	Description
EAI_UTIL_PROP_EAI_INIT_FLAG	Used to configure init flags to be used for eai_init for all instances. This property is set and supported only at eAI Utility init. - Default: 0 - struct: eai_util_eai_init_flag_t
EAI_UTIL_PROP_SCRATCH_MEM	Used to get/set the scratch memory, which is the maximum of all instances and used across all instances in a shared manner. - Valid only when re-use scratch is enabled - struct: eai_util_prop_scratch_mem_t
EAI_UTIL_PROP_MLA_USAGE	Used to set LPMLA usage - Default: Disabled - struct: eai_util_mla_usage_t
EAI_UTIL_INSTANCE_PROP_EAI_INIT_FLAG	Used to configure the init flags needed by eai_init for the given eAI instance. This property is set and supported only at eAI Utility init. - Default: 0 - Once set, this overrides the EAI_UTIL_PROP_EAI_INIT_FLAG. - struct: eai_util_instance_eai_init_flag_t
EAI_UTIL_INSTANCE_PROP_MLA_USAGE	Used to configure MLA for the given eAI instance - Default: 0 - struct: eai_util_instance_mla_usage_t
EAI_UTIL_MLA_CLIENT_INFO	Used to set LPMLA client information - Must be provided before MLA_Usage is set to TRUE - struct: eai_util_mla_client_info_t
EAI_UTIL_PROP_MLA_OFFLOAD_INFO	Used to get offload status to LPMLA struct: eai_util_mla_offload_t
EAI_UTIL_PROP_MLA_CONFIGURE_LOW_POWER_MODES	Used to let the LPMLA driver configure low power modes - Driver configures low power if TRUE, and removes configuration when FALSE - struct: eai_util_mla_configure_low_power_modes_t
EAI_UTIL_PROP_ENABLE_IO_BUF_REG	Used to enable eAI buf registration. - Enabled by default unless client disables it - When enabled, the following assumptions are made in eAI Utility: - Client queries all input and output tensor buffers from the eAI using EAI_PROP_TENSOR_INFO. eAI Utility registers the buffers when the client queries for the buffer and deregisters when scratch changes. When scratch changes, the client is expected to re-query the buffers. - If this behavior is not the expectation from the client, the client needs to disable the property. In this case, the client is responsible for registration and deregistration of each input/output buffers. - struct: eai_util_enable_io_buf_reg_t

4.2.1.2 eai_util_reuse_scratch_t

Data structure used to determine whether or not to reuse scratch memory across all eAI instances.

Parameter	Description
enable_scratch_reuse	If true, reuse scratch memory across all eAI instances hosted by the eAI Utility

4.2.1.3 eai_util_eai_init_flag_t

Data structure used to set generic init flags for `eai_init` function call.

Parameter	Description
eai_init_flags	Flags to be used for <code>eai_init</code> for all instances. Refer to the <code>EAI_INIT_FLAGS_XXX</code> definitions in the Hexagon SDK eAI documentation for possible values.

4.2.1.4 eai_util_prop_scratch_mem_t

Data structure used to get/set the scratch memory requirements of eAI Utility wrapper.

Parameter	Description
scratch_size	Scratch memory required for all eAI instances hosted by the eAI Utility
scratch_ptr	Scratch pointer provided by the client for the eAI instances hosted by the eAI Utility. Valid only for <code>set_prop()</code> .

4.2.1.5 eai_util_instance_eai_init_flag_t

Data structure used to set `eai_init_flags` for a specific instance. This should be passed in `init_prop_list` when calling the `eai_util_init()` function pointer to initialize a new eAI instance.

Parameter	Description
eai_instance_init_flags	Init flags to be used for a particular eAI instance. Refer to the <code>EAI_INIT_FLAGS_XXX</code> definitions in the Hexagon SDK eAI documentation for possible values.

4.2.1.6 eai_util_mla_usage_t

Data structure used to determine whether or not to use machine learning hardware for all eAI instances.

Parameter	Description
use_mla_hw_flag	0 – Do not use machine learning hardware 1 – Use machine learning hardware

4.2.1.7 eai_util_instance_mla_usage_t

Data structure used to determine whether or not to use machine learning hardware for a specific instance. This should be passed in `init_prop_list` when calling the `eai_util_init()` function pointer to initialize a new eAI instance.

Parameter	Description
eai_instance_mla_usage	0 – Do not use machine learning hardware 1 – Use machine learning hardware

4.2.1.8 eai_util_mla_client_info_t

Data structure used to set MLA client real-time processing information. Based on these parameters, the machine learning hardware driver votes its bandwidth and clock requirements.

Parameter	Description
frt_ratio	Faster than real-time processing request in multiples of 0.1 factors. For example, 10 = 1.0, 23 = 2.3, 35 = 3.5, etc.
frames_per_second	Number of frames per second

4.2.1.9 eai_util_mla_offload_t

Data structure to store property information.

Parameter	Description
offload_ratio	Percentage offload of total jobs offloaded to eNPU. The number needs to be divided by 100 by the client. Valid only when <code>enpu_version > 0</code> 0 – No offloading to eNPU 100 – All jobs are offloaded to eNPU
enpu_version	Offloaded eNPU version. 0 – No eNPU 1 – eNPU v1 2 – eNPU v2
reserved	Reserved for future use, should be set to 0 for now.

NOTE: To run the ADSP at the optimum clock, the algorithm/module developer should vote for proper Q6 KPPS according to how much load is offloaded to MLA. One way to identify this is to run your eAI algorithm in `eai_sample_app` (available in the Hexagon SDK at `addons\eAI\esai_runtime\`) with and without MLA offloading and check the cycle count in the simulation result. This should give a fair idea on algorithm computational requirements with and without MLA. This information can then be used to calculate overall the KPPS requirement of the algorithm with and without MLA.

4.2.1.10 eai_util_prop_t

Data structure to store property information.

Parameter	Description
prop_id	Property ID of type eai_util_prop
payload_ptr	Payload pointer according to property. See Section <section_link>.

4.2.1.11 eai_prop_list_t

Data structure used to store a list of init properties. This is passed in `eai_util_create()` to set generic init properties for all eAI instances and in the `eai_util_init()` function pointer to specify instance-specific init properties. The instance-specific init properties passed via `eai_util_init()` will override generic init properties set via `eai_util_create()` function call.

Parameter	Description
num_properties	Number of properties in prop_list
prop_list	Property list

4.2.1.12 eai_util_mla_configure_low_power_modes_t

Data structure used to let LPMLA Driver configure low power modes.

Parameter	Description
configure	LPMLA Driver configures low power if TRUE, and removes configuration when FALSE

4.2.1.13 eai_util_enable_io_buf_reg_t

Data structure used to enable eAI buf registration.

Parameter	Description
enable_eai_io_buf_reg	Set as TRUE to enable io buf registration.

4.2.2 API definitions

4.2.2.1 eai_util_create

Creates and initializes a new instance of eAI util wrapper.

Prototype

```
eai_util_intf_t* eai_util_create(eai_prop_list_t *init_prop_list,
                                POSAL_HEAP_ID heap_id)
```

Parameters

Parameter	In/Out	Description
init_prop_list	in	List of init properties
heap_id	in	Heap ID to use for all memory allocation

Returns

- Success – Returns pointer to newly created eAI Utility instance of type `eai_util_intf_t`
- Failure – NULL

4.2.2.2 eai_util_destroy

Destroys an `eai_util_intf` instance and calls deinit for all models.

Prototype

```
void eai_util_destroy(eai_util_intf_t* eai_util_intf)
```

Parameters

Parameter	In/Out	Description
eai_util_intf	in	List of init properties.

Returns

N/A

4.2.3 Virtual function definitions

The `eai_util_intf_t` interface data structure contains a list of virtual function pointers implemented by the eAI Utility wrapper. These virtual utility functions call core eAI APIs underneath and provide an abstraction layer between the user application/module and the eAI runtime. This interface also provides the functionality to manage multiple instances of eAI running under a single instance of the eAI Utility wrapper.

4.2.3.1 Eaiuh

This pointer contains a handle for the eAI Utility instance.

Prototype

```
void *eaiuh
```


4.2.3.2 eai_util_init

Initializes/loads an instance of the eAI runtime and applies platform-specific configuration properties. The implementation calls `eai_init`, `eai_set_property` (for heap ID, MLA, NPU usage, scratch usage, etc.), `eai_preapply`, and `eai_apply`.

Prototype

```
EAI_RESULT (*eai_util_init)(void *, uint32_t, void *, uint32_t,
eai_prop_list_t *)
```

Parameters

Parameter	In/Out	Description
eaiuh	in	eAI utility handle.
model_id	in	Identifier for model representing this eAI runtime instance. If model ID has already been initialized by the eAI utility, the associated eAI runtime must be deinit, then reinitialized with the new eai_graph.
eai_graph	in	Buffer containing eAI graph.
eai_buf_size	in	Size of buffer containing the eAI graph.
eai_prop_list_t *	in	Instance-specific list of init properties.

Returns

EAI_RESULT enum

4.2.3.3 eai_util_deinit

Deinitializes/unloads an instance of the eAI runtime. The implementation calls `eai_deinit` and cleans up any other resources for this eAI runtime instance.

Prototype

```
EAI_RESULT (*eai_util_deinit)(void *, uint32_t)
```

Parameters

Parameter	In/Out	Description
eaiuh	in	eAI Utility handle
model_id	in	Identifier for the model representing this eAI runtime instance

Returns

EAI_RESULT enum

4.2.3.4 eai_util_execute

Performs eAI graph execution. The implementation calls `eai_execute`.

Prototype

```
EAI_RESULT (*eai_util_execute)(void *, uint32_t, eai_batch_info_t *,
uint32_t)
```

Parameters

Parameter	In/Out	Description
eaiuh	in	eAI Utility handle
instance_id	in	Identifier for instance representing this eAI runtime instance.
batch_inputs	in	Array of batch information structures describing the execution request.
num_batches	in	Number of batch structures associated with the batch_inputs parameter.

Returns

EAI_RESULT enum

4.2.3.5 eai_util_get_property

Retrieves various settings. The implementation also calls `eai_get_property` for eAI runtime-specific properties. This property information is `eai_util_prop` type-specific. Refer to the `eai_util_prop` enumeration information to cross-correlate the required property information data types per property types.

Prototype

```
EAI_RESULT (*eai_util_get_property)(void *, uint32_t, uint32_t prop, void *prop_info)
```

Parameters

Parameter	In/Out	Description
eaiuh	in	eAI Utility handle
instance_id	in	Identifier for instance representing this eAI runtime instance.
prop	in	Identifies the property type being retrieved.
prop_info	out	Identifies the property type information being retrieved.

Returns

EAI_RESULT enum

4.2.3.6 eai_util_set_property

Function to set various settings. The implementation also calls `eai_set_property` for eAI runtime-specific properties. This property information is `eai_util_prop` type-specific. Refer to the `eai_util_prop` enumeration information to cross-correlate the required property information data types per property types.

Prototype

```
EAI_RESULT (*eai_util_set_property)(void *, uint32_t, uint32_t prop, void *prop_info)
```

Parameters

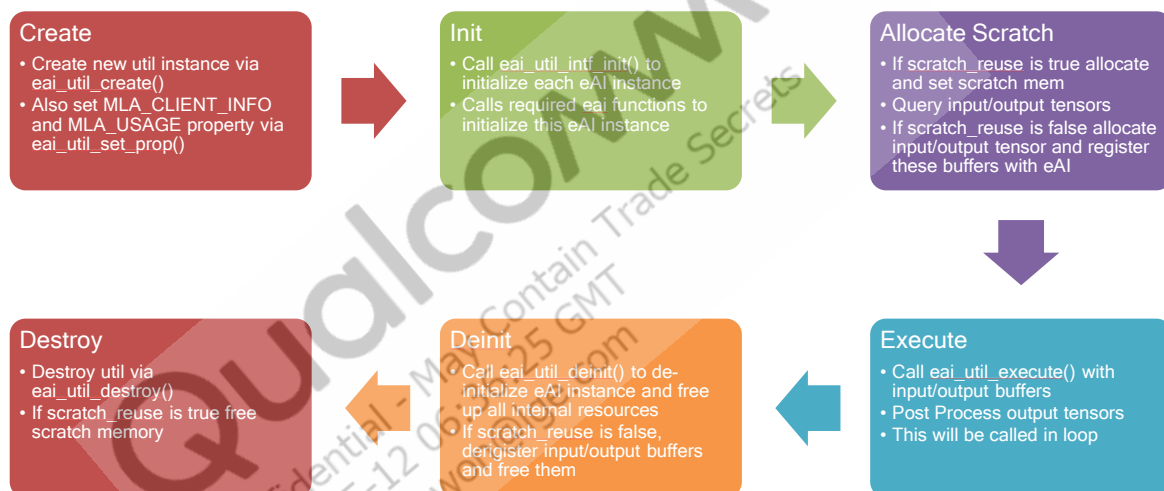
Parameter	In/Out	Description
eaiuh	in	eAI Utility handle
instance_id	in	Identifier for instance representing this eAI runtime instance.
prop	in	Identifies the property type being set.
prop_info	out	Identifies the property type information being set.

Returns

EAI_RESULT enum

4.3 eAI Utility call flow

The following diagram describes top level call flow of eAI Utility.



From this top level diagram, the call flow is mainly divided into the following parts. Each stage is described in detail in subsequent sections.

- Initialization
- IO buffer registration
- Execute graph
- De-initialization

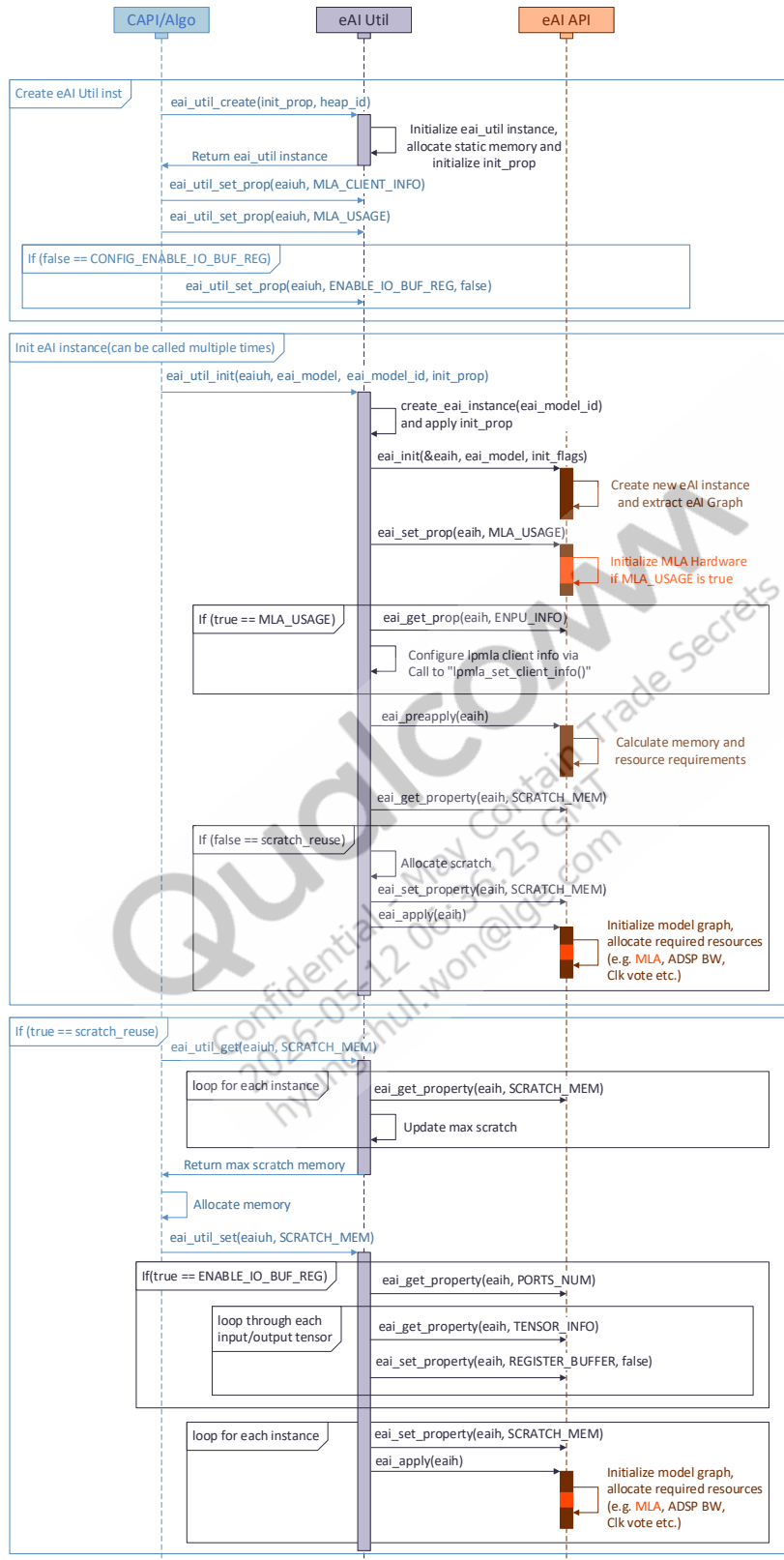
4.3.1 Initialization

Initialization consists of following steps:

- eAI wrapper instance initialization – Done once in any module/algorithm
- Once the wrapper is initialized, multiple eAI instances can be initialized based on the number of model modules/algorithms needed to be loaded
- If `scratch_reuse` is True, the module/algorithm needs to query the required scratch memory to the eAI Utility wrapper, allocate the required scratch memory, and provide this scratch memory to the eAI Utility wrapper. If `scratch_reuse` is False, the module/algorithm does not need to worry about allocating this scratch memory as the eAI Utility wrapper will allocate the required memory and provide it to the eAI runtime.
- On newer chipsets, the eAI runtime framework is now part of Root PD. So, if the module/algorithm developer is allocating input/output buffer on their own, these buffers need to be registered and deregistered by the module/algorithm developer to share its access between Audio and Root PD.

The module/algorithm developer needs to carefully choose whether or not to enable `scratch_reuse`. When `scratch_reuse` is True, all eAI instances running under the same wrapper will use the same piece of scratch memory, which will allow the module/algorithm to run with a lower overall memory footprint. But here, the module/algorithm developer needs to make sure that any calls to `eai_util_execute` are made sequentially for all eAI instances, resulting in only one `eai_util_execute` being executed at any given point of time.

The following diagram shows detailed a level call flow for initialization. It shows how the eAI Utility wrapper APIs convert into eAI runtime APIs.



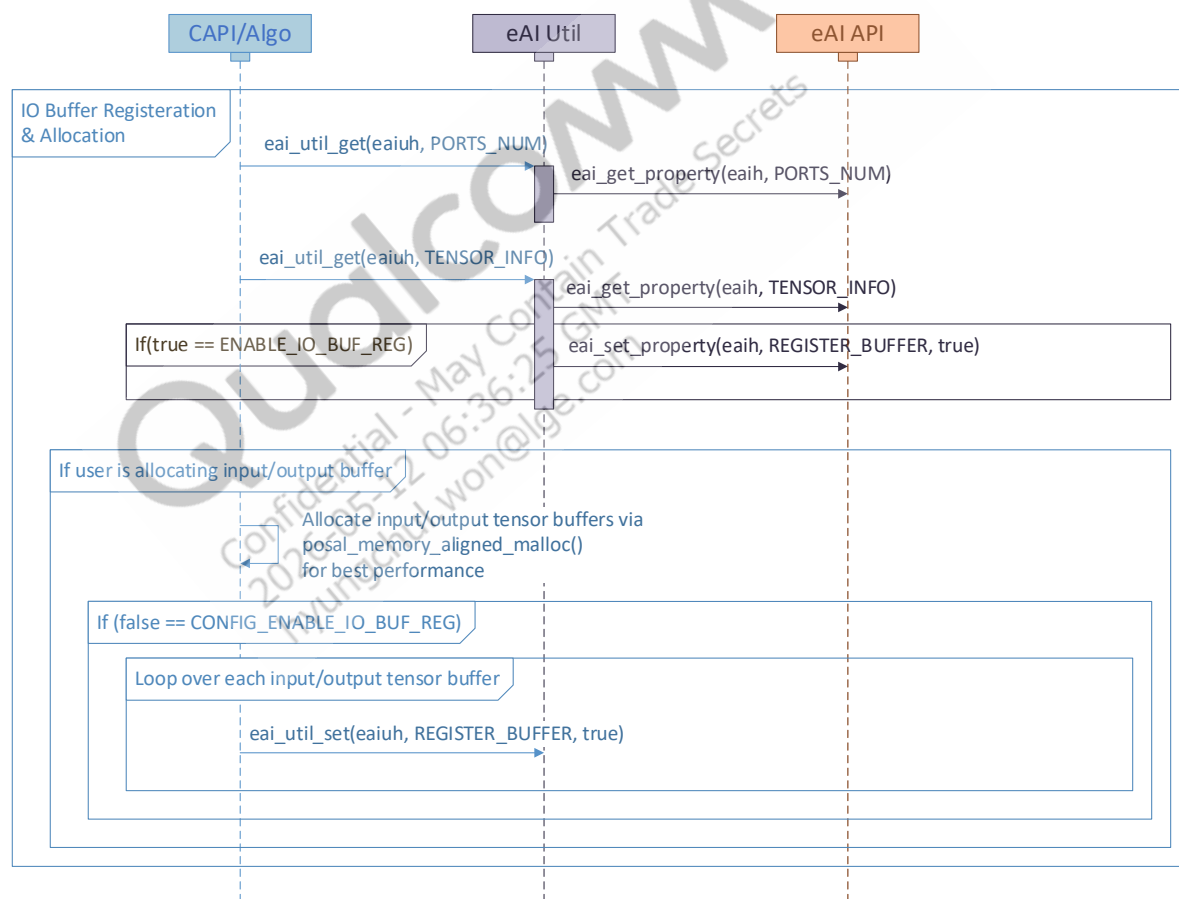
4.3.2 IO buffer registration

On new chipsets, the eAI runtime framework is now part of Root PD. So, the module/algorithm developer must register all input/output buffers via `EAI_PROP_REGISTER_BUFFER` eAI runtime property. `eai_util` provides the functionality to do this registration/deregistration on its own via the `EAI_UTIL_PROP_ENABLE_IO_BUF_REG` property.

The default value of the `EAI_UTIL_PROP_ENABLE_IO_BUF_REG` property is True, and it assumes that the client queries all input and output tensor buffers from the eAI using `EAI_PROP_TENSOR_INFO`. eAI utility registers the buffers when the client queries for the buffer and deregisters when scratch changes. When scratch changes, the client is expected to re-query the buffers.

If the client does not query all input/output tensor via the `EAI_PROP_TENSOR_INFO` property, then the client needs to disable the property. In this case, the client is responsible for registration and deregistration of each input/output buffer.

The following call flow explains the buffer registration process.

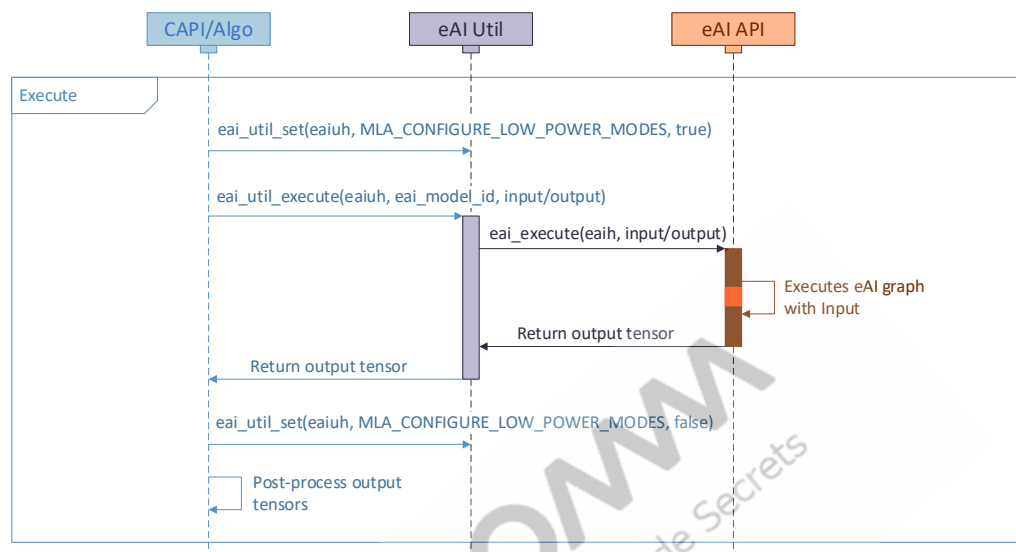


The module/algorithm developer can either allocate input/output tensors on their own or can choose to reuse the input/output tensors allocated by the eAI framework. If the module/algorithm developer chooses to allocate memory for input/output tensors on their own, the developer must ensure all allocated memory is 256 byte aligned for optimum performance.

4.3.3 Execute

Once initialization and buffer registration are done, the execute step is called repeatedly for as long as the module/algorithm has data to process.

The following diagram shows a detailed level call flow for execution. It shows how the eAI Utility wrapper APIs convert into eAI runtime APIs.

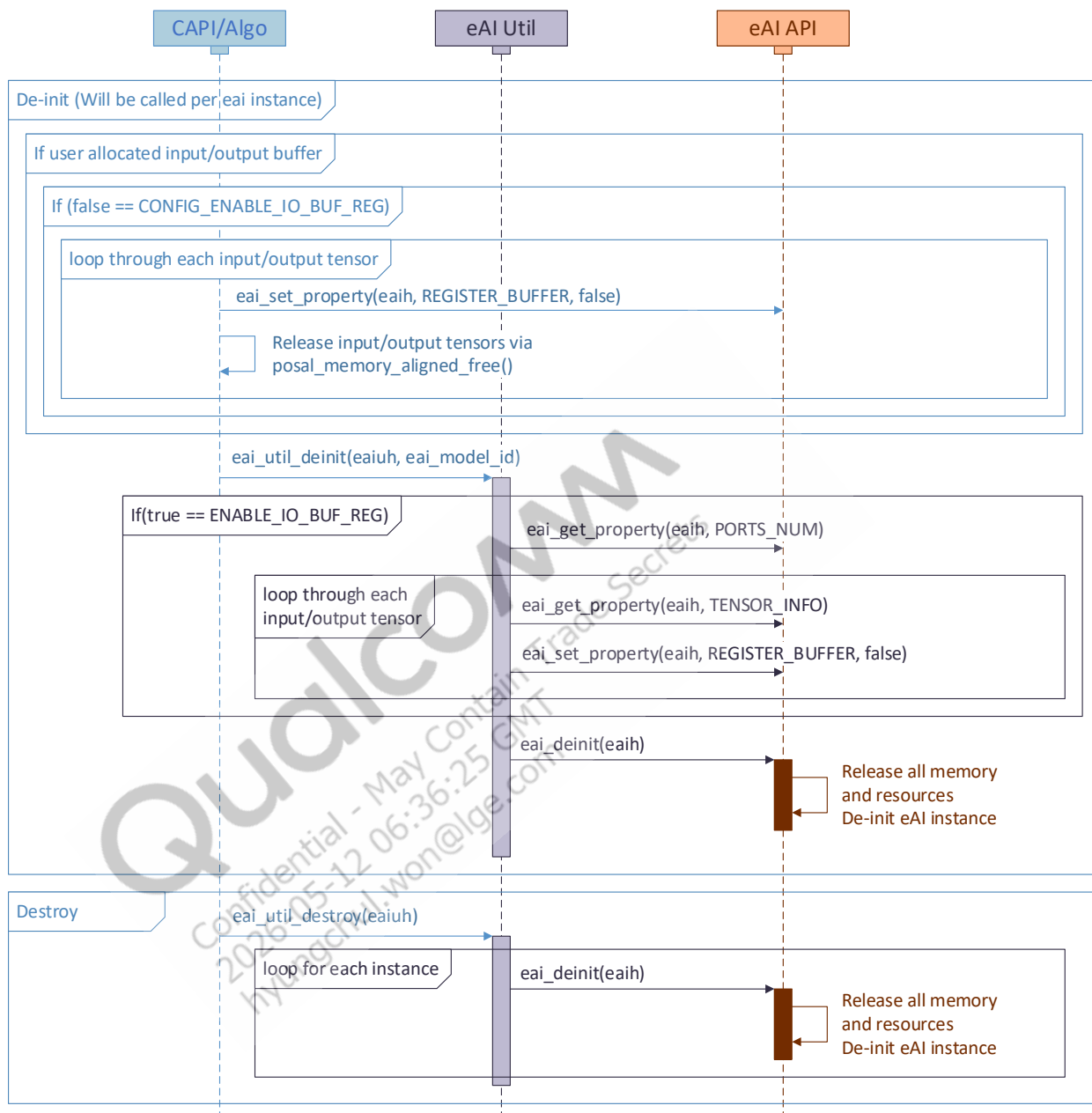


4.3.4 De-initialization

Once all data is processed, the module/algorithm developer needs to call the de-initialization routine to free up all memory allocated by the eAI Utility wrapper and eAI runtime.

If the module/algorithm developer allocated input/output buffers on their own, these buffers need to be de-registered first. Then the module/algorithm can call `eai_util_deinit` per eAI instance to de-initialize these eAI instances. Once all eAI instances are de-allocated, the module/algorithm developer can destroy the eAI Utility wrapper to de-allocate eAI Utility wrapper memory.

The following diagram shows a detailed level call flow for the de-initialization sequence. It shows how eAI Utility wrapper APIs convert into eAI runtime APIs.



5 Example

5.1 Building and running a CAPI-based eAI example detection module on-target

5.1.1 Build a CAPI module in the ADSP image

This section describes how to compile the `detection_engine_eai` example CAPI module in an ADSP build.

5.1.1.1 Scons file changes for static island and non-island compilation

In the module's main scons file (`detection_engine_eai.scons`, located at `adsp_proc/avs/fw/oem/detection_engine_eai/build/`), add “_island” as shown to compile the static library for island:

```
if 'USES_AVS_SHARED_LIBS' in env:
    SRCPATH='..'
    env.VariantDir('${BUILDPATH}', SRCPATH, duplicate=0)
    env.AddAvsSharedLibrary(['AVS_SHARED'], '${BUILDPATH}/${LIBNAME}',
                            [avs_reg_sources, avs_stub_sources,
                             avs_prop_sources], build_type='MOD')
else:
    if 'USES_AUDIO_IN_ISLAND' in env:
        get_island_lib = env.AddAvsLibrary(['AVS_ADSP', 'AVS_ADSP_USER'],
                                             '${BUILDPATH}/${LIBNAME}+_island', [avs_reg_sources,
                                             avs_stub_sources, avs_prop_sources], 'MOD')

    if 'USES_AVS_LIBRARY_BUILDER' in env:
        x = env.AddAvsLibrary(['AVS_ADSP', 'AVS_ADSP_USER'],
                              '${BUILDPATH}/${LIBNAME}', [avs_reg_sources, avs_stub_sources,
                              avs_prop_sources], 'MOD')
```

To compile the static library in non-island, remove the “_island” suffix:

```
get_island_lib = env.AddAvsLibrary(['AVS_ADSP', 'AVS_ADSP_USER'],
                                     '${BUILDPATH}/${LIBNAME}', [avs_reg_sources,
                                     avs_stub_sources, avs_prop_sources], 'MOD')
```

For shared library generation, no changes are required in the scons file.

5.1.1.2 Add the library in the json file

Add the library in the pp_libs_cfg.json file present at adsp_proc\avs\build\<target>\ in the ADSP build.

In the following example, detection_engine_eai is statically integrated in the ADSP in LPI mode.

```
{
  "lib_name"           : "detection_engine_eai",
  "is_needed_for_hy22" : false,
  "build"              : "STATIC_BUILD_NO_STRIP",
  "lib_major_ver"      : 1,
  "lib_minor_ver"      : 0,
  "island"             : "True",
  "is_qurt_used"        : true,
  "amdb_info" :
  {
    "itype"             : "capi",
    "mtype"             : "generic",
    "mid"               : "0x1700109D",
    "tag"               : "capi_detection_engine_eai",
    "module_name"       : "MODULE_ID_DETECTION_ENGINE_EAI"
  }
},
```

Similarly, to integrate the use case in Non-LPI mode, change the following field in the json file:

```
"island" : "False",
```

To integrate the module as dynamic, change the following field in the json file:

```
"build" : "SHARED_BUILD_NO_STRIP",
```

5.1.1.3 Build ADSP images

Run the following commands to build the ADSP image with the new changes:

```
cd <ADSP_BUILD_DIR>\adsp_proc\build\ms
python build_variant.py <target>.adsp.prod
```

5.1.2 Build the SoundTriggerTestApp from the LA build

The SoundTriggerTest App is present in the LA build and can be compiled via the following commands:

```
$ cd <LA_UM_ROOT_DIR>
$ source build/envsetup.sh
$ lunch <target>-userdebug
$ cd frameworks/base/tests/SoundTriggerTestApp/
$ mm
```

Once the compilation is completed successfully, SoundTriggerTestApp.apk should be present at <LA_UM_ROOT_DIR>/out/target/product/<target>/system/priv-app/SoundTriggerTestApp/.

5.1.3 Update resourcemanager.xml file with custom VA config

Add the following section in resourcemanager_qrd.xml file based on the target.

```
<!-- CUSTOM VA Config-->
<sound_model_config>
  <param vendor_uuid="ba2d19c7-931f-4f21-9b73-f9ab3c41b93e"/>
  <param get_module_version="false" />
  <param kw_start_tolerance="100" />
  <param kw_end_tolerance="100" />
  <param merge_first_stage_sound_models="false"/>
  <param capture_keyword="2000" />
  <param client_capture_read_delay="2000" />
  <!-- Profile specific data which the algorithm can support -->
  <param sample_rate="16000" />
  <param bit_width="16" />
  <param out_channels="1"/> <!-- Module output channels -->
  <param capture_keyword="2000" />
  <param client_capture_read_delay="2000" />
  <operating_modes>
    <low_power
capture_profile_handset="SINGLE_MIC_16KHZ_16BIT_RAW_LPI"
capture_profile_headset="SINGLE_MIC_16KHZ_16BIT_HEADSET_RAW_LPI"/>
    <high_performance
capture_profile_handset="DUAL_MIC_16KHZ_16BIT_RAW_NLPI"
capture_profile_headset="SINGLE_MIC_16KHZ_16BIT_HEADSET_RAW_NLPI"/>
    <high_performance_and_charging
capture_profile_handset="DUAL_MIC_16KHZ_16BIT_RAW_NLPI"
capture_profile_headset="SINGLE_MIC_16KHZ_16BIT_HEADSET_RAW_NLPI"/>
  </operating_modes>
  <module_params>
    <param load_sound_model_ids="0xC0000008, 0x0800104C" />
    <param unload_sound_model_ids="0xC0000008, 0x00012C15" />
    <param custom_config_ids="0xC0000008, 0x00012C20" />
```

```

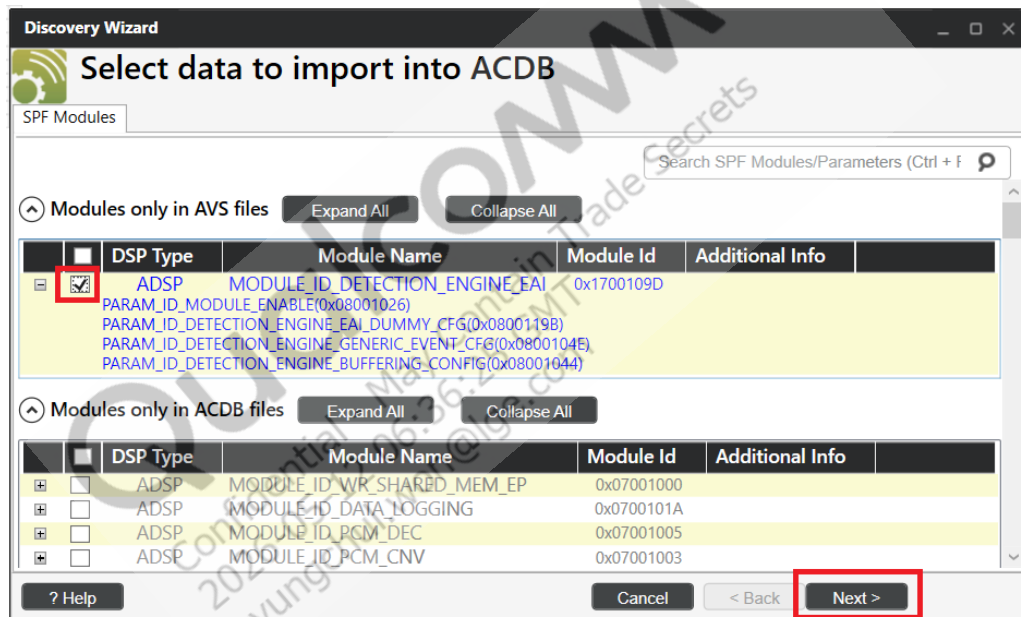
<param buffering_config_ids="0xC0000008, 0x08001044" />
<param version_ids="0xC0000008, 0x18000101" />
<param engine_reset_ids="0xC0000008, 0x08001051" />
<streamConfig-metadata>
  <!-- refer kvh2xml.h -->
  <kvpair key="0xBC000000" value="0xBC000004"/>
</streamConfig-metadata>
</module_params>
</sound_model_config>

```

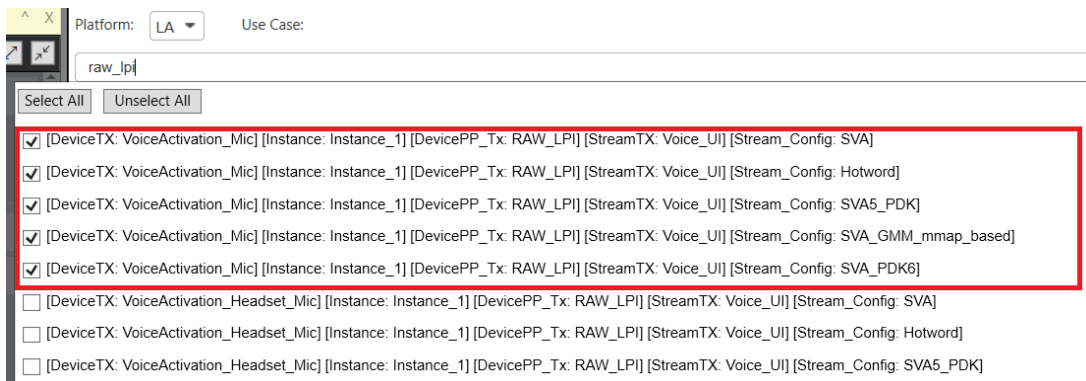
5.1.4 Update ACDB files to include eAI-based CAPI module

This section describes how to add a new Voice Activation use case in the STREAM_CFG_VUI_CUSTOM stream type. For more details, refer to *Low Power Island Voice Activation in AudioReach* (80-VN500-20).

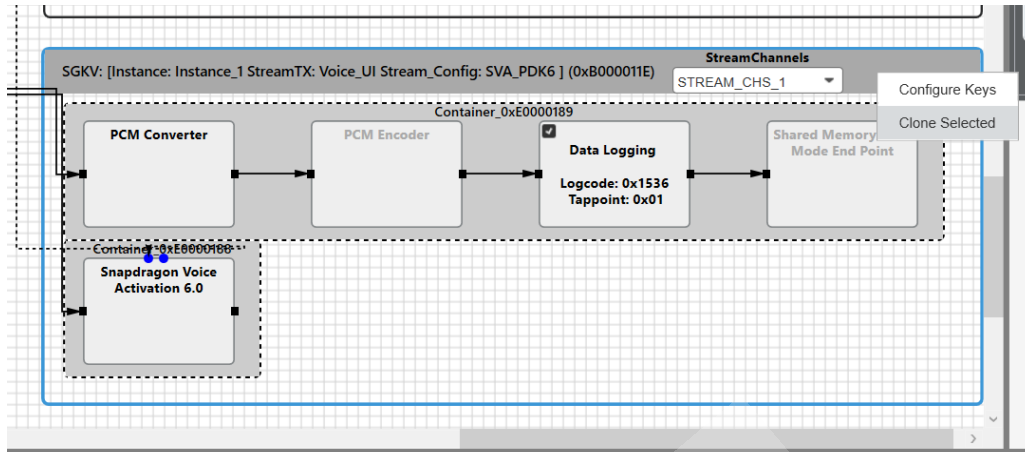
1. Import a custom VA module to QACT™ via the Module Discovery Wizard.



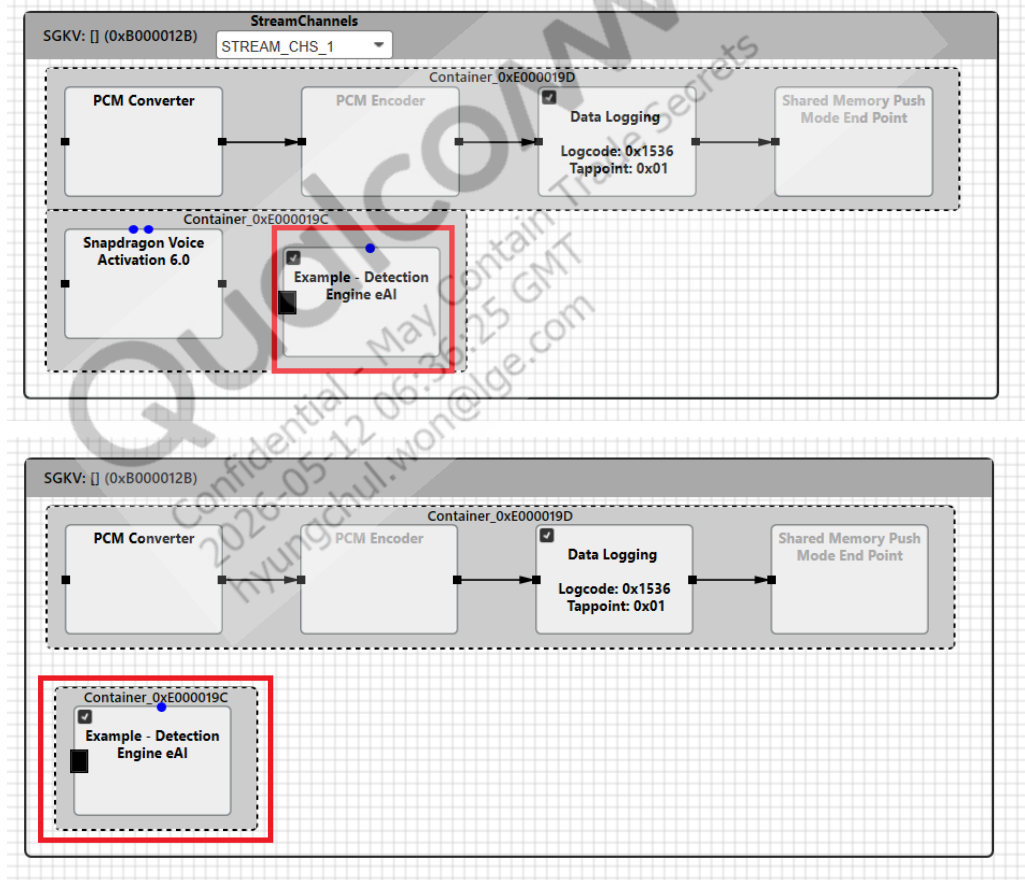
2. Select all the graphs with [Device Tx: VoiceActivation_Mic] and [DevicePP_Tx: RAW_LPI] configuration as shown:



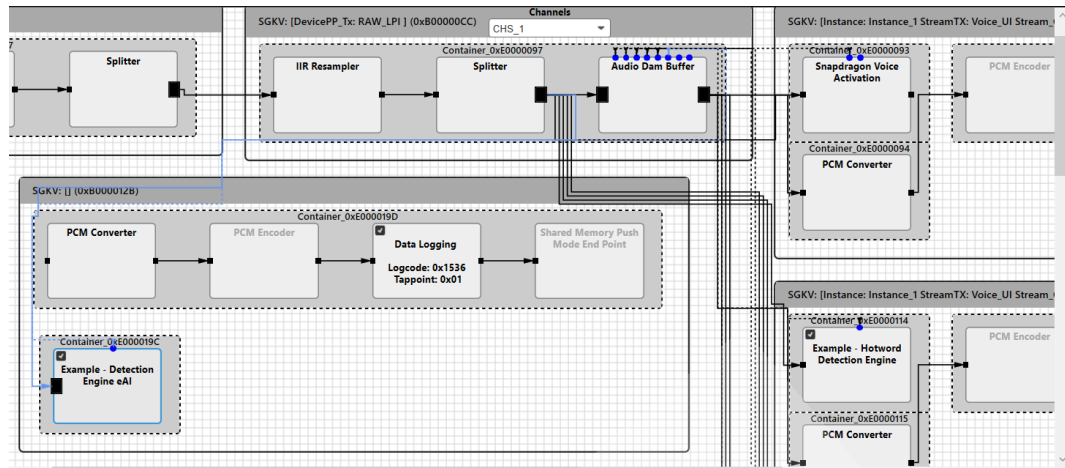
3. Click **Start Graph Modification** and clone any of the Stream Tx graph as shown:



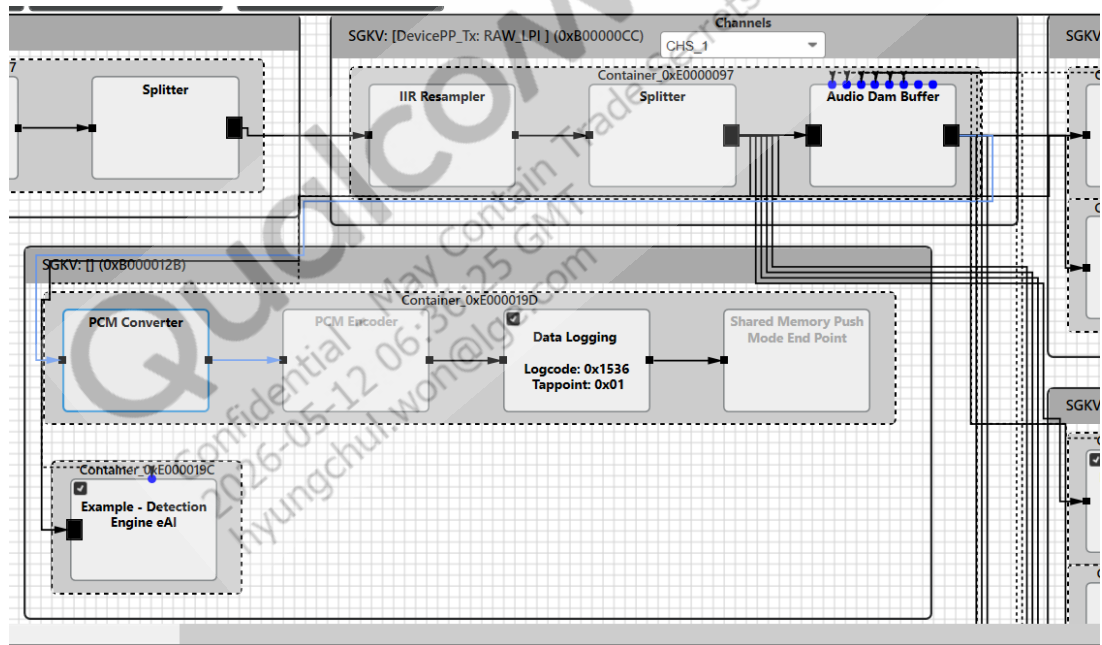
4. Add the eAI example module in same container where the Voice Activation module is present. Once the eAI example module is added in the container, delete the Voice Activation module from this container.



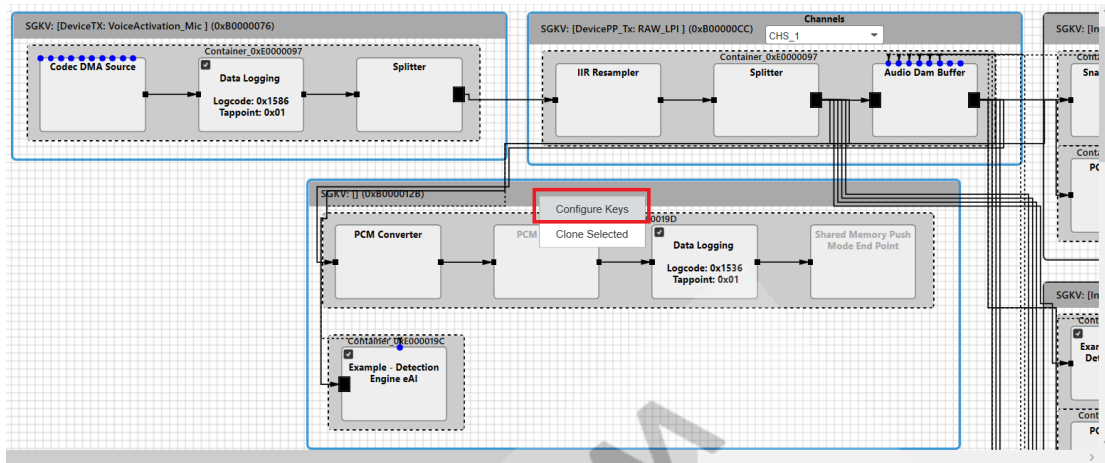
5. Connect the eAI example module's input with the Splitter of the [DevicePP_Tx: RAW_LPI] subgraph and connect one of the available Audio Dam Buffer module's control links to the eAI example module's control link as shown:



6. Connect the Audio Dam Buffer output to the PCM Converter module of the subgraph.

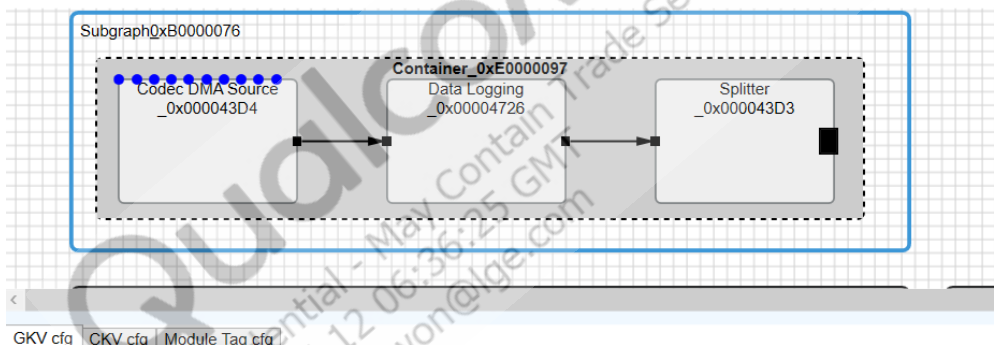


7. Select all three subgraphs – [Device Tx: VoiceActivation_Mic], [DevicePP_Tx: RAW_LPI] and the newly cloned subgraph.
8. Right click and select **Configure Keys**.
9. Click **Yes** to create a new GKV for this new graph.



10. In the Key Configurator window, assign SGKVs and tags as shown:

1. Select subgraph(s) or module(s) to configure:



2. Select Keys for subgraph(s):

Select	Key	ID
☐	StreamPP_RX	0xAF000000
☐	StreamPP_TX	0xB0000000
☐	Instance	0xAB000000
☐	VSID	0xB3000000
☐	DevicePP_Rx	0xAC000000
☐	BtProfile	0xB4000000
☐	SW_Sidetone	0xBA000000
☐	Stream	0xD3000000
☐	ProxyTxType	0xD1000000
☒	DeviceTX	0xA3000000
☐	BtFormat	0xB5000000
☐	StreamRX	0xA1000000

3. Configure Key Value for selected SubGraphs(s):

Set SGKV

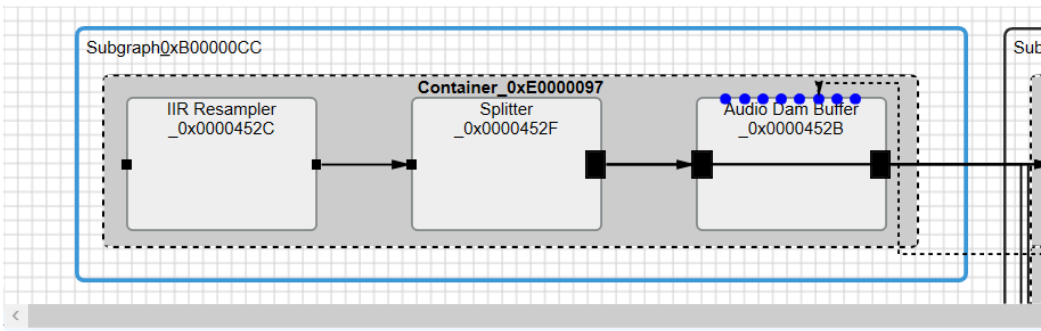
DeviceTX

- ☐ Speaker_Mic
- ☐ BT_Tx
- ☐ Headphone_Mic
- ☐ Handset_Mic
- ☐ USB_Tx
- ☒ VoiceActivation_Mic
- ☐ VoiceActivation_Headset_Mic
- ☐ Proxy_Tx
- ☐ VI_TX
- ☐ FM_TX

4. SubGraphs and SubGraph Key Vectors:

SubGraph	SubGraph Key Vector
0xB0000076	

1. Select subgraph(s) or module(s) to configure:



GKV cfg CKV cfg Module Tag cfg

2. Select Keys for subgraph(s):

Select	Key	ID
☐	DevicePP_Rx	0xAC000000
☐	BTProfile	0xB4000000
☐	SW_Sidetone	0xBA000000
☐	Stream	0xD3000000
☐	ProxyTxType	0xD1000000
☐	DeviceTX	0xA3000000
☐	BTFormat	0xB5000000
☐	StreamRX	0xA1000000
☐	DeviceRX	0xA2000000
☒	DevicePP_Tx	0xAD000000
☐	StreamTX	0xB1000000
☐	Stream_Config	0xBC000000

3. Configure Key Value for selected SubGraphs(s):

Set SGKV

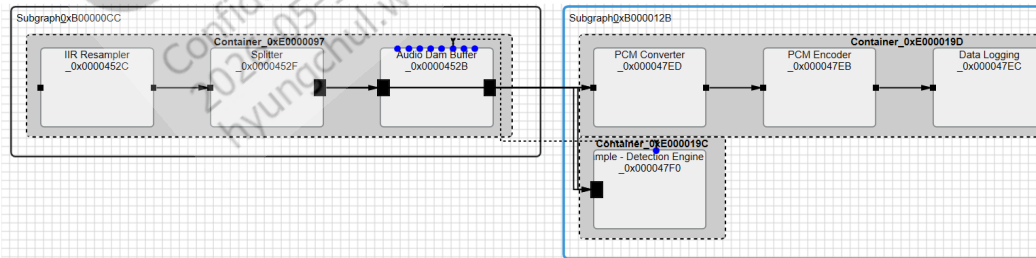
DevicePPTx

- ☐ HFP_Sink_Fluence_SMECNS
- ☐ Voip_Fluence_SMECNS
- ☐ Voice_Fluence_SMECNS
- ☐ Voice_Fluence_Endfire
- ☐ Voice_Fluence_Pro
- ☐ Fluence_FFNS
- ☒ RAW_LPI
- ☐ Voip_Fluence_Endfire
- ☐ RAW_NLPI
- ☐ Voice_Fluence_NN_SM

4. SubGraphs and SubGraph Key Vectors:

SubGraph	SubGraph Key Vector
Subgraph0xB00000CC	0xB0000076 0xB00000CC 0xB000012B

1. Select subgraph(s) or module(s) to configure:



GKV cfg CKV cfg Module Tag cfg

2. Select Keys for subgraph(s):

Select	Key	ID
☐	DevicePP_Rx	0xAC000000
☐	BTProfile	0xB4000000
☐	SW_Sidetone	0xBA000000
☐	Stream	0xD3000000
☐	ProxyTxType	0xD1000000
☐	DeviceTX	0xA3000000
☐	BTFormat	0xB5000000
☐	StreamRX	0xA1000000
☐	DeviceRX	0xA2000000
☐	DevicePP_Tx	0xAD000000
☒	StreamTX	0xB1000000
☒	Stream_Config	0xBC000000

3. Configure Key Value for selected SubGraphs(s):

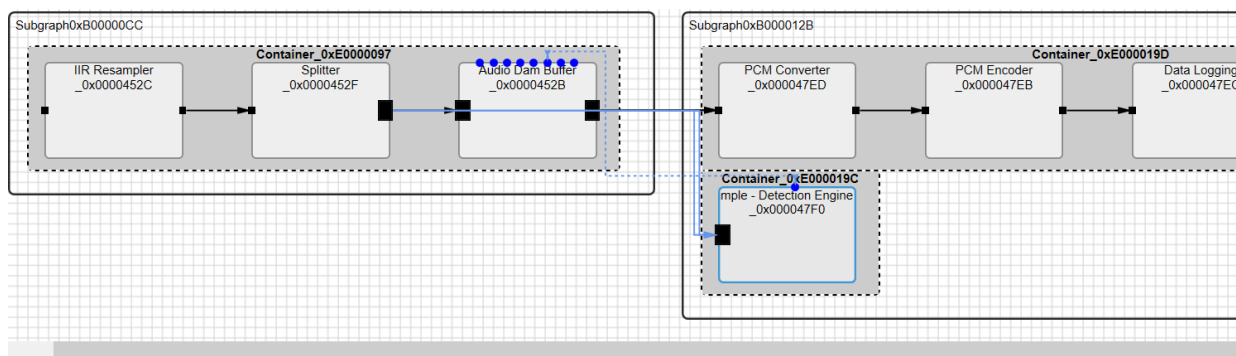
Set SGKV

StreamTX

- ☐ PCM_Record
- ☐ PCM_Tx_Loopback
- ☒ Voice_UI
- ☐ Voip_Tx
- ☐ HFP_Rx_Capture
- ☐ HFP_Tx_Capture
- ☐ Voice_Call_Tx
- ☐ DeepBuffer_Record
- ☐ RAW_Record
- ☐ PCM_ULL_Record

StreamConfig

- ☐ SVA
- ☐ Hotword
- ☐ SVA5_PDK
- ☒ CUSTOM
- ☐ SVA_GMM_mmap_based
- ☐ SVA_PDK6
- ☐ ACD_QC



3KV cfg | CKV cfg | Module Tag cfg

2. Please config PIDs:

Module	PID	Name
Example - Detection Engine eAI	0x8001026	PARAM_ID_MODULE_ENABLE
Example - Detection Engine eAI	0x800119B	PARAM_ID_DETECTION_ENGINE_E
Example - Detection Engine eAI	0x800104E	PARAM_ID_DETECTION_ENGINE_G
Example - Detection Engine eAI	0x8001044	PARAM_ID_DETECTION_ENGINE_B

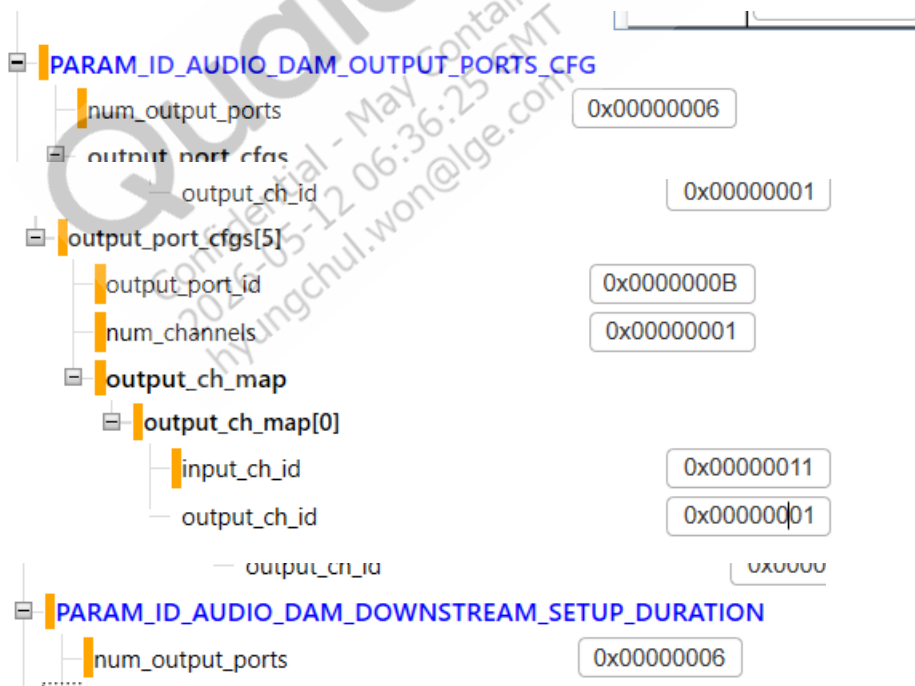
3. Select Tags for Module(s):

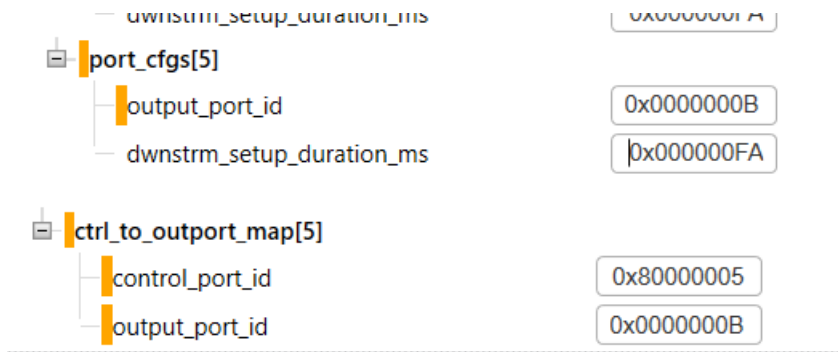
Tag	ID
☐ sh_ep	0xC0000001
☐ device_mfc	0xC000000C
☐ stream_volume	0xC000000D
☐ stream_pcm_decoder	0xC000000E
☐ stream_pcm_encoder	0xC000000F
☐ stream_pcm_converter	0xC0000010
☐ stream_pause	0xC0000006
☐ stream_mute	0xC0000007
☐ stream_input_media_format	0xC0000002
☐ stream_output_media_format	0xC0000003
☒ device_sva	0xC0000008
☐ device_adam	0xC0000009

4. Configure TKV for selected Module(s):

Add a Tag/TKV

11. Double click on the Audio Dam Buffer module and update the new connections information as shown:





12. Save the ACDB files.

5.1.5 Run end-to-end use case on target

To run the end-to-end use case on the target:

1. Push the updated ACDB files to the target:

```
$ adb wait-for-device shell rm /vendor/etc/acdbdata/QRD/*
$ adb push QRD_acdb_cal.acdb /vendor/etc/acdbdata/QRD
$ adb push QRD_workspaceFileXml.qwsp /vendor/etc/acdbdata/QRD
```

2. Push resourcemanager.xml to the target:

```
$ adb push resourcemanager_qrd.xml /vendor/etc/resourcemanager_qrd.xml
```

3. Install the SoundTriggerTestApp on the target:

```
$ adb install -r SoundTriggerTestApp.apk
```

4. Open the SoundTriggerTestApp and allow microphone permission to the SoundTriggerTestApp.

5. Create an `eai_custom.properties` file with the following content:

```
name=eAI-Custom
dataFile=org_sample_model_fixed.eai
vendorUuid=ba2d19c7-931f-4f21-9b73-f9ab3c41b93e
modelUuid=ba2d19c7-931f-4f21-9b73-f9ab3c41b93e
```

NOTE: Make sure there are no trailing spaces in the above content. `vendorUuid` and `modelUuid` should match with the uuid provided in `resourcemanager.xml` file.

6. Push the properties and model file to the target:

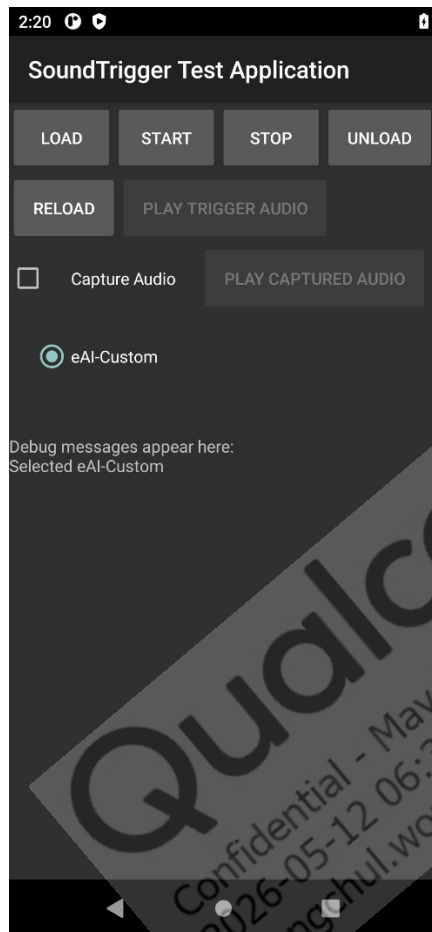
```
$ adb push org_sample_model_fixed.eai
/data/data/com.android.test.soundtrigger/files/
$ adb push eai_custom.properties
/data/data/com.android.test.soundtrigger/files/
```

NOTE: This `org_sample_model_fixed.eai` model is available in the Hexagon SDK eAI add-on.

- Flash the updated ADSP images. The device will automatically reboot after flashing.

```
$ cd <ADSP_BUILD_DIR>/adsp_proc/config/scripts/  
$ python flashadsp.py
```

- Once the device reboots, launch the SoundTriggerTestApp. It should launch with the following UI:



- Click **Load** and **Start** to start the VA use case with the custom topology. Once the model is loaded, it should trigger a dummy detection event after some time.
- To stop the use case, click **Stop** and **Unload**.

5.2 CAPI example overview

This section explains the eAI-related code of the detection_engine_eai example module.

5.2.1 Data structures and enumerations

In order to make various function calls from the CAPI wrapper, some data structures must be initialized. The following are the definitions of the required data structures.

```
// Macro definitions

#define MAX_MODEL_INOUT_COUNT      1
#define DUMMY_EAI_MODEL_ID        0x00001234
#define DEFAULT_MLA_USAGE          1
#define DEFAULT_SCRATCH_REUSE      1
#define DEFAULT_ENABLE_IO_BUF_REG  0
#define DEFAULT_IO_MEMORY_ALIGNMENT 256
/* Faster than real time processing request: multiples of 0.1 factors
 * Eg: 10 = 1.0, 23 = 2.3, 35 = 3.5, etc */
#define MLA_FTRT_RATIO             10
#define MLA_FRAMES_PER_SECOND     100          // 100 = 10ms frame size

/* Detection module debug message */
#define MEAI_MSG_PREFIX "module_eai_utils[0x%X]: "
#define MEAI_DBG(ID, xx_ss_mask, xx_fmt, ...) AR_MSG(xx_ss_mask,
EAI_MSG_PREFIX xx_fmt, ID, ##__VA_ARGS__)

#define MEAI_DBG_ISLAND(ID, xx_ss_mask, xx_fmt, ...) \
    AR_MSG_ISLAND(xx_ss_mask, MEAI_MSG_PREFIX xx_fmt, ID, ##__VA_ARGS__)

#define GET_MODULE_EAI_STATE(p_eai_data) ((p_eai_data)->state)
#define GET_MODULE_EAI_MODEL_SIZE(p_eai_data) ((p_eai_data)->model_size)

// Enumerations type definitions
typedef enum module_eai_result {
    MODULE_EAI_RET_SUCCESS = 0,
    MODULE_EAI_RET_FAIL,
    MODULE_EAI_RET_BAD_PARAM,
    MODULE_EAI_RET_NULL_REF,
    MODULE_EAI_RET_NOMEM,
    MODULE_EAI_RET_BAD_STATE,
```

```
} module_eai_result_t;

typedef enum module_eai_state {
    MODULE_EAI_STATE_NULL = 0,
    MODULE_EAI_STATE_INIT,
    MODULE_EAI_STATE_MODEL_RVCD,
    MODULE_EAI_STATE_MODEL_LOADED,
    MODULE_EAI_STATE_READY,
} module_eai_state_t;

// Data structure definitions
typedef struct eai_data_t {
    uint32_t heap_id;
    /* scratch Memory requirement of the algo*/
    eai_util_intf_t* p_eai_util_intf;
    uint32_t mla_enable;
    uint32_t enable_io_buf_reg;
    module_eai_state_t state;

    uint32_t reuse_scratch;
    uint8_t *scratch_mem;
    uint32_t scratch_mem_size;

    bool_t is_model_persistent;
    uint8_t *model;
    uint32_t model_id;
    uint32_t model_size;

    size_t input_count;
    size_t input_size[MAX_MODEL_INOUT_COUNT];
    uint32_t input_type[MAX_MODEL_INOUT_COUNT];
    uint8_t *input_addr[MAX_MODEL_INOUT_COUNT];

    size_t output_count;
    size_t output_size[MAX_MODEL_INOUT_COUNT];
    uint32_t output_type[MAX_MODEL_INOUT_COUNT];
    uint8_t *output_addr[MAX_MODEL_INOUT_COUNT];

    eai_batch_info_t batch_info;
```

```

    eai_buffer_info_t inputs_buffer_info[MAX_MODEL_INOUT_COUNT];
    eai_buffer_info_t outputs_buffer_info[MAX_MODEL_INOUT_COUNT];
} eai_data_t;

```

Most of the definitions, enums, and data structures are self-explanatory. The `eai_data_t` data structure stores all parameters needed for the eAI Utility and its input/output tensor. Currently, it assumes a maximum of one input and output tensor, but this can be changed via the `MAX_MODEL_INOUT_COUNT` macro.

5.2.2 Initialization

The `module_eai_init()` function explains how to create an `eai_util` instance. This function does the following:

- Checks the model state. If it is in the NULL state (`MODULE_EAI_STATE_NULL`), it continues. Otherwise, it returns `MODULE_EAI_RET_BAD_STATE`.
- Initialize the `eai_data_t` data structure
- Take the `heap_id`, `mla_usage` and `reuse_scratch` variables from the user and store their values accordingly in the `eai_data_t` data structure
- Create and initialize the `eai_util` instance
- Set the `MLA_CLIENT_INFO` and `MLA_USAGE_INFO` properties of `eai_util`
- Disable IO buffer registration if `scratch_reuse` is False and `DEFAULT_ENABLE_IO_BUF_REG` is 0

The `module_eai_init()` function can be called from the CAPI module's init function callback.

```

/* Create and initialize eAI util instance */
module_eai_result_t module_eai_init(eai_data_t *p_eai_data, uint32_t
heap_id, uint32_t mla_usage, uint32_t reuse_scratch)
{
    EAI_RESULT = EAI_SUCCESS;
    module_eai_result_t result = MODULE_EAI_RET_SUCCESS;
    eai_util_intf_t *p_util_intf = NULL;
    eai_util_eai_init_flag_t init_flags;
    eai_util_reuse_scratch_t eai_reuse_scratch;

    if (!p_eai_data) {
        MEAI_DBG(p_eai_data, DBG_ERROR_PRIO, "Invalid input parameter");
        return MODULE_EAI_RET_NULL_REF;
    }

    if (MODULE_EAI_STATE_NULL != p_eai_data->state) {
        MEAI_DBG(p_eai_data, DBG_ERROR_PRIO, "Cur state = %d, expected

```

```

        MODULE_EAI_STATE_NULL", p_eai_data->state);
    MEAI_DBG(p_eai_data, DBG_ERROR_PRIO,
        "Cur state = %d, expected MODULE_EAI_STATE_NULL",
        p_eai_data->state);
    return MODULE_EAI_RET_BAD_STATE;
}

memset(p_eai_data, 0x00, sizeof(eai_data_t));
p_eai_data->model_id = -1;

p_eai_data->heap_id = heap_id;
p_eai_data->m1a_enable = m1a_usage;
p_eai_data->reuse_scratch = reuse_scratch;
MEAI_DBG(p_eai_data, DBG_ERROR_PRIO, "m1a_enable = %d,
    scratch_reuse = %d", p_eai_data->m1a_enable,
    p_eai_data->reuse_scratch);
/* Initialization of eAI util instance */
/* Set the init flag if EAI needs to allocate memory in island heap*/
init_flags.eai_init_flags = (p_eai_data->heap_id == POSAL_HEAP_DEFAULT)
    ? 0 : EAI_INIT_FLAGS_LPI;
/* Systems Team Recommended to enable scratch memory reuse for SVA5*/
eai_reuse_scratch.enable_scratch_reuse = reuse_scratch;
eai_util_prop_t eai_prop[] = {{ EAI_UTIL_PROP_REUSE_SCRATCH_MEM,
    (int8_t *) &eai_reuse_scratch },
    { EAI_UTIL_PROP_EAI_INIT_FLAG,
    (int8_t *) &init_flags }};
eai_prop_list_t eai_init_prop_list = {
    sizeof(eai_prop) / sizeof(eai_util_prop_t), eai_prop};
};

/* Creating EAI Utility */
p_util_intf = eai_util_create(&eai_init_prop_list,
    p_eai_data->heap_id);
if (NULL == p_util_intf) {
    MEAI_DBG(p_eai_data, DBG_ERROR_PRIO, "Error in creating
        eai_util_intf");
    result = MODULE_EAI_RET_FAIL;
    goto bail;
}
/* set required properties of eai util wrapper */

```

```

/* set MLA CLIENT INFO */
eai_util_mla_client_info_t eai_util_mla_client_info;
eai_util_mla_client_info.ftrt_ratio = MLA_FTRT_RATIO;
eai_util_mla_client_info.frames_per_second = MLA_FRAMES_PER_SECOND;
eai_result = p_util_intf->eai_util_set_property(p_util_intf->eaiuh, 0,
                                              EAI_UTIL_MLA_CLIENT_INFO,
                                              (void *)(&eai_util_mla_client_info));

if (eai_result != EAI_SUCCESS) {
    MEAI_DBG(p_eai_data, DBG_ERROR_PRIO, "EAI_UTIL_MLA_CLIENT_INFO Set
        failed, result:0x%x", eai_result);
    result = MODULE_EAI_RET_FAIL;
    goto bail;
}

/* set MLA USAGE INFO */
eai_util_mla_usage_t eai_util_mla_usage;
eai_util_mla_usage.use_mla_hw_flag = p_eai_data->mla_enable;
eai_result = p_util_intf->eai_util_set_property(p_util_intf->eaiuh, 0,
                                              EAI_UTIL_PROP_MLA_USAGE,
                                              (void*) (&eai_util_mla_usage));

if (eai_result != EAI_SUCCESS) {
    MEAI_DBG(p_eai_data, DBG_ERROR_PRIO, "EAI_UTIL_PROP_MLA_USAGE Set
        failed, result:0x%x", eai_result);
    result = MODULE_EAI_RET_FAIL;
    goto bail;
}

/* Disable IO buffer registration if scratch_reuse is false
 * and DEFAULT_ENABLE_IO_BUF_REG is 0.
 * Note: IO Buffer registration is independent of scratch_reuse
 * Property. But in this example we only allocate input/output buffers
 * if scratch_reuse is false so we are using scratch_reuse here */

if ((FALSE == p_eai_data->reuse_scratch) &&
    (0 == DEFAULT_ENABLE_IO_BUF_REG)) {
    p_eai_data->enable_io_buf_reg = 0;
    eai_util_enable_io_buf_reg_t io_buffer_reg = {
        p_eai_data->enable_io_buf_reg };
    eai_result = p_util_intf->eai_util_set_property(
        p_util_intf->eaiuh, 0,

```



```

        EAI_UTIL_PROP_ENABLE_IO_BUF_REG,
        (void *) (&io_buffer_reg));

    if (eai_result != EAI_SUCCESS) {
        MEAI_DBG(p_eai_data, DBG_ERROR_PRIO,
            "EAI_UTIL_PROP_ENABLE_IO_BUF_REG Set failed,
            result:0x%x", eai_result);
        result = MODULE_EAI_RET_FAIL;
        goto bail;
    }
} else {
    /* This is default value in utility, so no need to set in util */
    p_eai_data->enable_io_buf_reg = 1;
}

/* Cache the EAI Utility Instance pointer to be passed while calling
   EAI Destroy */
p_eai_data->p_eai_util_intf = p_util_intf;

/* Update state to initialized */
p_eai_data->state = MODULE_EAI_STATE_INIT;
MEAI_DBG(p_eai_data, DBG_HIGH_PRIO, "Initialized successfully");
return MODULE_EAI_RET_SUCCESS;

bail:
    if (p_util_intf) {
        eai_util_destroy(p_util_intf);
        p_util_intf = NULL;
    }
    if (p_eai_data) {
        memset(p_eai_data, 0x00, sizeof(eai_data_t));
    }
    /* Update state to NULL */
    p_eai_data->state = MODULE_EAI_STATE_NULL;
    MEAI_DBG(p_eai_data, DBG_ERROR_PRIO, "Fail to Initialize, error = %d",
        result);
    return result;
}

```

Once `eai_util` is successfully created, the eAI model can be loaded in the eAI runtime. The following helper functions are used to initialize/load/store the eAI Utility instance:

- `module_eai_store_and_load_model()` – Stores and loads a new eAI model. It does the following :
 - Checks the model state. If it is in `MODULE_EAI_STATE_INIT` state it continues. Otherwise, it returns `MODULE_EAI_RET_BAD_STATE`.
 - If model state is `MODULE_EAI_STATE_INIT`, it stores a new eAI model via `module_eai_store_model()` helper function
 - Unloads the previous model if any model is already loaded
 - After storing the model successfully, it loads the model via `module_eai_load_model()` helper function
- `module_eai_store_model()` – Stores a new eAI model. It does the following:
 - Checks the model state. If it is in `MODULE_EAI_STATE_INIT` state, it continues. Otherwise, it returns `MODULE_EAI_RET_BAD_STATE`.
 - If the `is_model_persistent` flag is set as `FALSE`, it allocates model memory and copies the model in `eai_data_t` data structure. Otherwise, it does nothing.
 - After storing the eAI model successfully, it changes the model state to `MODULE_EAI_STATE_MODEL_RVCD`
- `module_eai_load_model()` – Loads a new eAI model. It does the following:
 - Checks the model state. If it is in `MODULE_EAI_STATE_MODEL_RVCD` state, it continues. Otherwise, it returns `MODULE_EAI_RET_BAD_STATE`.
 - If model state is `MODULE_EAI_STATE_MODEL_RVCD`, it calls `eai_util_init()` to initialize the eAI instances with the given model.
 - After loading the model successfully, it updates the model state to `MODULE_EAI_STATE_MODEL_LOADED`
 - If the `reuse_scratch` flag is set as `TRUE`, it also gets the scratch memory required from the eAI runtime via the `eai_util_get_property()` function call and allocates the required scratch memory, and then sets it via the `eai_util_set_property()` function call.
 - Once the eAI model is initialized in `eai_util`, it queries input/output tensor related information and updates `eai_data_t` accordingly via a helper function `module_eai_update_inout_tensors()`.
 - If anything goes wrong while storing/loading the model, this function unloads the model and returns an error
 - After loading the model successfully, it updates the model state to `MODULE_EAI_STATE_READY`
- `module_eai_update_inout_tensors()` – Gets input/output tensors information. It does the following:

- Parses the number of input and output ports and their respective sizes via `module_eai_get_eai_port_info()` function call
- Gets each input tensor information via `eai_util_get_property()` function call and updates the `eai_data_t` data structure
- If the `reuse_scratch` flag is set as `FALSE`, it allocates the input and output tensor memory and updates the `eai_data_t` data structure accordingly. If anything goes wrong, this function releases all input/output tensor related memory allocated in the `eai_data_t` data structure via `reset_input_output_info()`.
- If `enable_io_buf_reg` is `FALSE`, then this function will also register allocated input/output buffers to eAI via `EAI_PROP_REGISTER_BUFFER` set prop.
- `module_eai_get_eai_port_info()` – Static function to get input/output port numbers and tensor size information via `eai_util_get_property()` function call
- `reset_input_output_info()` – Releases all input/output tensor related memory allocated in the `eai_data_t` data structure.

```

/* Helper functions */
/* To store and load a new eAI model */
module_eai_result_t module_eai_store_and_load_model(eai_data_t *p_eai_data,
                                                    uint32_t model_id, uint8_t *model_ptr, uint32_t model_size)
{
    module_eai_result_t result = MODULE_EAI_RET_SUCCESS;
    if (!p_eai_data) {
        MEAI_DBG(p_eai_data, DBG_ERROR_PRIO, "Invalid input arguments");
        return MODULE_EAI_RET_BAD_PARAM;
    }
    MEAI_DBG(p_eai_data, DBG_HIGH_PRIO, "In %s...", __func__);
    if (MODULE_EAI_STATE_INIT != p_eai_data->state) {
        MEAI_DBG(p_eai_data, DBG_ERROR_PRIO, "Cur state = %d, expected\nMODULE_EAI_STATE_INIT", p_eai_data->state);
        return MODULE_EAI_RET_BAD_STATE;
    }
    /* Store new eAI model */
    result = module_eai_store_model(p_eai_data, model_id, model_ptr,
                                    model_size);
    if (MODULE_EAI_RET_SUCCESS != result) {
        MEAI_DBG(p_eai_data, DBG_ERROR_PRIO, "Store eAI model failed, err\n= %d", result);
        return MODULE_EAI_RET_FAIL;
    }
}

```

```

/* Load eAI model in eAI framework */
MEAI_DBG(p_eai_data, DBG_HIGH_PRIO, "Loading eAI Model");
result = module_eai_load_model(p_eai_data);

if (MODULE_EAI_RET_SUCCESS != result) {
    MEAI_DBG(p_eai_data, DBG_ERROR_PRIO, "Fail to load eAI model,
        err = %d", result);
    MEAI_DBG(p_eai_data, DBG_HIGH_PRIO, "Releasing eAI model...");
    result = module_eai_release_model(p_eai_data);
    if (MODULE_EAI_RET_SUCCESS != result) {
        MEAI_DBG(p_eai_data, DBG_ERROR_PRIO, "Fail to release eAI
            model, err = %d", result);
    }
    return MODULE_EAI_RET_FAIL;
}
return result;
}

/* To store a new eAI model */
module_eai_result_t module_eai_store_model(eai_data_t *p_eai_data,
    uint32_t model_id, uint8_t *model_ptr, uint32_t model_size)
{
    if (!p_eai_data || !model_ptr || !model_size) {
        MEAI_DBG(p_eai_data, DBG_ERROR_PRIO, "Invalid input arguments");
        return MODULE_EAI_RET_BAD_PARAM;
    }
    if (MODULE_EAI_STATE_INIT != p_eai_data->state) {
        MEAI_DBG(p_eai_data, DBG_ERROR_PRIO, "Cur state = %d, expected
            MODULE_EAI_STATE_INIT", p_eai_data->state);
        return MODULE_EAI_RET_BAD_STATE;
    }
    if (p_eai_data->is_model_persistent) {
        /* Model is persistent, do not allocate memory for model */
        p_eai_data->model = model_ptr;
    } else {
        /* Allocate model memory and copy the model */

        p_eai_data->model = (uint8_t *)posal_memory_malloc(model_size,
            p_eai_data->heap_id);
        memcpy(p_eai_data->model, model_ptr, model_size);
    }
}

```

```

    }

    p_eai_data->model_size = model_size;
    p_eai_data->model_id = model_id;
    /* Change the state to model received */
    p_eai_data->state = MODULE_EAI_STATE_MODEL_RVCD;
    return MODULE_EAI_RET_SUCCESS;
}

/* To load a new eAI model */
module_eai_result_t module_eai_load_model(eai_data_t *p_eai_data)
{
    eai_util_intf_t *p_util_intf;
    EAI_RESULT = EAI_SUCCESS;
    module_eai_result_t result = MODULE_EAI_RET_SUCCESS;
    if (!p_eai_data) {
        MEAI_DBG(p_eai_data, DBG_ERROR_PRIO, "Invalid input arguments");
        return MODULE_EAI_RET_BAD_PARAM;
    }

    if (MODULE_EAI_STATE_MODEL_RVCD != p_eai_data->state) {
        MEAI_DBG(p_eai_data, DBG_ERROR_PRIO, "Cur state = %d, expected\n",
                MODULE_EAI_STATE_MODEL_RVCD, p_eai_data->state);
        return MODULE_EAI_RET_BAD_STATE;
    }
    p_util_intf = p_eai_data->p_eai_util_intf;
    MEAI_DBG(p_eai_data, DBG_HIGH_PRIO, "Calling eai_util_init with eaiuh\n",
            = 0x%08p, model_id = %d, ...", p_util_intf->eaiuh,
            p_eai_data->model_id);
    eai_result = p_util_intf->eai_util_init(p_util_intf->eaiuh,
                                           p_eai_data->model_id,
                                           p_eai_data->model,
                                           p_eai_data->model_size,
                                           NULL);

    if (EAI_SUCCESS != eai_result) {
        MEAI_DBG(p_eai_data, DBG_ERROR_PRIO, "eai_util_init failed with\n",
                error %d", eai_result);
        result = MODULE_EAI_RET_FAIL;
        goto bail;
    }
}

```

```

    }

    MEAI_DBG(p_eai_data, DBG_HIGH_PRIO, "eai_util_init completed
        successfully.");

    /* Model loaded successfully, move the loaded state now, so if something
        fails here after we know we have to unload the model */
    p_eai_data->state = MODULE_EAI_STATE_MODEL_LOADED;

    /* Allocate scratch memory if reuse_scratch is true */
    if (TRUE == p_eai_data->reuse_scratch) {
        eai_util_prop_scratch_mem_t prop_info = { 0 };
        eai_result = p_util_intf->eai_util_get_property(p_util_intf->eaiuh,
            0, EAI_UTIL_PROP_SCRATCH_MEM, &prop_info);
        if (eai_result != EAI_SUCCESS) {
            MEAI_DBG(p_eai_data, DBG_ERROR_PRIO, "Fail to get
                EAI_UTIL_PROP_SCRATCH_MEM(%d), error = %d",
                EAI_UTIL_PROP_SCRATCH_MEM, eai_result);
            result = MODULE_EAI_RET_FAIL;
            goto bail;
        }
        p_eai_data->scratch_mem_size = prop_info.scratch_size;
        MEAI_DBG(p_eai_data, DBG_HIGH_PRIO, "Allocating %u bytes of scratch
            memory", p_eai_data->scratch_mem_size);
        p_eai_data->scratch_mem = (uint8_t *)posal_memory_malloc(
            p_eai_data->scratch_mem_size, p_eai_data->heap_id);
        prop_info.scratch_ptr = (int8_t *)p_eai_data->scratch_mem;
        eai_result = p_util_intf->eai_util_set_property(p_util_intf->eaiuh,
            0, EAI_UTIL_PROP_SCRATCH_MEM, &prop_info);
        if (eai_result != EAI_SUCCESS) {
            MEAI_DBG(p_eai_data, DBG_ERROR_PRIO, "Fail to set
                EAI_UTIL_PROP_SCRATCH_MEM(%d)" , error = %d",
                EAI_UTIL_PROP_SCRATCH_MEM, eai_result);
            result = MODULE_EAI_RET_FAIL;
            goto bail;
        }
        MEAI_DBG(p_eai_data, DBG_HIGH_PRIO, "Scratch memory updated
            successfully in eai_util");
    }

    result = module_eai_update_inout_tensors(p_eai_data);

```

```

    if (MODULE_EAI_RET_SUCCESS != result) {
        MEAI_DBG(p_eai_data, DBG_ERROR_PRIO, "Fail to setup input/output
            tensor, error = %d", result);
        result = MODULE_EAI_RET_FAIL;
        goto bail;
    }
    /* we can process the data now, move to ready state */
    p_eai_data->state = MODULE_EAI_STATE_READY;
    return MODULE_EAI_RET_SUCCESS;
bail:
    if (MODULE_EAI_STATE_MODEL_LOADED == p_eai_data->state) {
        /* Unload the model */
        MEAI_DBG(p_eai_data, DBG_ERROR_PRIO, "Load model failed, unloading
            eAI model");
        result = module_eai_unload_model(p_eai_data);
        if (MODULE_EAI_RET_SUCCESS != result) {
            MEAI_DBG(p_eai_data, DBG_ERROR_PRIO, "Fail to unload model,
                error %d", result);
            return MODULE_EAI_RET_FAIL;
        }
    }
    return result;
}

/* To Query input/output tensor information*/
module_eai_result_t module_eai_update_inout_tensors(eai_data_t *p_eai_data)
{
    size_t i;
    eai_util_intf_t *p_util_intf;
    eai_tensor_info_t tensor_info;
    EAI_RESULT = EAI_SUCCESS;
    module_eai_result_t result = MODULE_EAI_RET_SUCCESS;
    eai_register_buffer_t buf_info = { NULL, 0,
                                        EAI_REG_BUF_FLAGS_REGISTER };
    eai_register_buffer_info_t buf_prop = { 1, &buf_info };
    if (!p_eai_data) {
        MEAI_DBG(p_eai_data, DBG_ERROR_PRIO, "Invalid input parameter");
        return MODULE_EAI_RET_NULL_REF;
    }
    p_util_intf = p_eai_data->p_eai_util_intf;

```

```

// Get number of input ports and their sizes
result = module_eai_get_eai_port_info(p_eai_data, p_eai_data->model_id,
    0, &(p_eai_data->input_count), p_eai_data->input_size);
if (result != MODULE_EAI_RET_SUCCESS) {
    MEAI_DBG(p_eai_data, DBG_ERROR_PRIO, "Fail to get Input port
        details, error %d", result);
    result = MODULE_EAI_RET_FAIL;
    goto bail;
}
// Get each input tensor information
tensor_info.input_or_output = 0;
for (i = 0; i < p_eai_data->input_count; i++) {
    tensor_info.index = (uint32_t) i;
    eai_result = p_util_intf->eai_util_get_property(p_util_intf->eaiuh,
        p_eai_data->model_id, EAI_PROP_TENSOR_INFO, &tensor_info);
    if (eai_result != EAI_SUCCESS) {
        MEAI_DBG(p_eai_data, DBG_ERROR_PRIO, "Fail to get eai property
            EAI_PROP_TENSOR_INFO(%d)", EAI_PROP_TENSOR_INFO);
        result = MODULE_EAI_RET_FAIL;
        goto bail;
    }
    p_eai_data->input_type[i] = tensor_info.element_type;
    p_eai_data->input_addr[i] = tensor_info.address;
    MEAI_DBG(p_eai_data, DBG_HIGH_PRIO, "%s Tensor %d Info:",
        tensor_info.input_or_output?"Output":"Input",
        tensor_info.index);
    MEAI_DBG(p_eai_data, DBG_HIGH_PRIO, "  Tensor address      :
        0x%08p", tensor_info.address);
    MEAI_DBG(p_eai_data, DBG_HIGH_PRIO, "  Tensor element_type : %u",
        tensor_info.element_type);
    MEAI_DBG(p_eai_data, DBG_HIGH_PRIO, "  Tensor tensor_size  : %u",
        tensor_info.tensor_size);
    MEAI_DBG(p_eai_data, DBG_HIGH_PRIO, "  Tensor num_dims     : %u",
        tensor_info.num_dims);
    for (int j = 0; j < tensor_info.num_dims; j++) {
        MEAI_DBG(p_eai_data, DBG_HIGH_PRIO, "    [%d] dim %u,
            stride %u", j, tensor_info.dims[j],
            tensor_info.stride[j]);
    }
}

```



```

}
// Get number of output ports and their sizes
result = module_eai_get_eai_port_info(p_eai_data, p_eai_data->model_id,
    1, &(p_eai_data->output_count), p_eai_data->output_size);
if (result != MODULE_EAI_RET_SUCCESS) {
    MEAI_DBG(p_eai_data, DBG_ERROR_PRIO, "Fail to get Output port
        details, error %d", result);
    result = MODULE_EAI_RET_FAIL;
    goto bail;
}
// Get each output tensor information
tensor_info.input_or_output = 1;
for (i = 0; i < p_eai_data->output_count; i++) {
    tensor_info.index = (uint32_t) i;
    eai_result = p_util_intf->eai_util_get_property(p_util_intf->eaiuh,
        p_eai_data->model_id, EAI_PROP_TENSOR_INFO, &tensor_info);

    if (eai_result != EAI_SUCCESS) {
        MEAI_DBG(p_eai_data, DBG_ERROR_PRIO, "Fail to get eai property
            EAI_PROP_TENSOR_INFO(%d)", EAI_PROP_TENSOR_INFO);
        result = MODULE_EAI_RET_FAIL;
        goto bail;
    }
    p_eai_data->output_type[i] = tensor_info.element_type;
    p_eai_data->output_addr[i] = tensor_info.address;
    MEAI_DBG(p_eai_data, DBG_HIGH_PRIO, "%s Tensor %d Info:",
        tensor_info.input_or_output?"Output":"Input",
        tensor_info.index);
    MEAI_DBG(p_eai_data, DBG_HIGH_PRIO, "  Tensor address      :
        0x%08p", tensor_info.address);
    MEAI_DBG(p_eai_data, DBG_HIGH_PRIO, "  Tensor element_type : %u",
        tensor_info.element_type);
    MEAI_DBG(p_eai_data, DBG_HIGH_PRIO, "  Tensor tensor_size  : %u",
        tensor_info.tensor_size);
    MEAI_DBG(p_eai_data, DBG_HIGH_PRIO, "  Tensor num_dims     : %u",
        tensor_info.num_dims);
    for (int j = 0; j < tensor_info.num_dims; j++) {
        MEAI_DBG(p_eai_data, DBG_HIGH_PRIO, "      [%d] dim %u,

```

```

        stride %u", j, tensor_info.dims[j], tensor_info.stride[j]);
    }
}

// Allocate input/output buffers if scratch reuse if false
if (FALSE == p_eai_data->reuse_scratch) {
    for (i = 0; i < p_eai_data->input_count; i++) {
        p_eai_data->input_addr[i] =
            (uint8_t *) posal_memory_aligned_malloc (
                p_eai_data->input_size[i],
                DEFAULT_IO_MEMORY_ALIGNMENT,
                p_eai_data->heap_id);
        if (!p_eai_data->input_addr[i]) {
            MEAI_DBG(p_eai_data, DBG_ERROR_PRIO, "Fail to allocate
                memory for input buffer");
            result = MODULE_EAI_RET_FAIL;
            goto bail;
        }
        if (0 == p_eai_data->enable_io_buf_reg) {
            buf_info.buf = p_eai_data->input_addr[i];
            buf_info.buf_size = p_eai_data->input_size[i];
            eai_result = p_util_intf->eai_util_set_property(
                p_util_intf->eaiuh,
                p_eai_data->model_id,
                EAI_PROP_REGISTER_BUFFER,
                &buf_prop);
            if ((EAI_SUCCESS != eai_result) &&
                (EAI_UNSUPPORTED_PROP != eai_result)) {
                MEAI_DBG(p_eai_data, DBG_ERROR_PRIO,
                    "Fail to register io buffer, error %d",
                    eai_result);
                result = MODULE_EAI_RET_FAIL;
                goto bail;
            }
        }
    }
}

for (i = 0; i < p_eai_data->output_count; i++) {
    p_eai_data->output_addr[i] =
        (uint8_t *) posal_memory_aligned_malloc (

```

```

        p_eai_data->output_size[i],
        DEFAULT_IO_MEMORY_ALIGNMENT,
        p_eai_data->heap_id);
    if (!p_eai_data->output_addr[i]) {
        MEAI_DBG(p_eai_data, DBG_ERROR_PRIO, "Fail to allocate
            memory for input buffer");
        result = MODULE_EAI_RET_FAIL;
        goto bail;
    }
    if (0 == p_eai_data->enable_io_buf_reg) {
        buf_info.buf = p_eai_data->output_addr[i];
        buf_info.buf_size = p_eai_data->output_size[i];
        eai_result = p_util_intf->eai_util_set_property(
            p_util_intf->eaiuh,
            p_eai_data->model_id,
            EAI_PROP_REGISTER_BUFFER,
            &buf_prop);

        if ((EAI_SUCCESS != eai_result) &&
            (EAI_UNSUPPORTED_PROP != eai_result)) {
            MEAI_DBG(p_eai_data, DBG_ERROR_PRIO,
                "Fail to register io buffer, error %d",
                eai_result);
            result = MODULE_EAI_RET_FAIL;
            goto bail;
        }
    }
}

// set up buffer info and batch info for eAI
p_eai_data->batch_info.num_inputs = p_eai_data->input_count;
for (i = 0; i < p_eai_data->input_count; i++) {
    p_eai_data->inputs_buffer_info[i].index = i;
    p_eai_data->inputs_buffer_info[i].addr =
        p_eai_data->input_addr[i];
    p_eai_data->inputs_buffer_info[i].buffer_size =
        (uint32_t) p_eai_data->input_size[i];
    p_eai_data->inputs_buffer_info[i].element_type =
        (uint32_t) p_eai_data->input_type[i];
}

```

```

    }

    p_eai_data->batch_info.num_outputs = p_eai_data->output_count;
    for (i = 0; i < p_eai_data->output_count; i++) {
        p_eai_data->outputs_buffer_info[i].index = i;
        p_eai_data->outputs_buffer_info[i].addr =
            p_eai_data->output_addr[i];
        p_eai_data->outputs_buffer_info[i].buffer_size =
            (uint32_t) p_eai_data->output_size[i];
        p_eai_data->outputs_buffer_info[i].element_type =
            (uint32_t) p_eai_data->output_type[i];
    }

    p_eai_data->batch_info.inputs = p_eai_data->inputs_buffer_info;
    p_eai_data->batch_info.outputs = p_eai_data->outputs_buffer_info;
    return MODULE_EAI_RET_SUCCESS;

bail:
    /* Reset all input/output pointers */
    reset_input_output_info(p_eai_data);
    return result;
}

/* To input/output port numbers and tensor size information */
static module_eai_result_t module_eai_get_eai_port_info(eai_data_t
*p_eai_data, uint32_t model_id, uint32_t input_or_output, size_t *count,
size_t *sizes)
{
    EAI_RESULT = EAI_SUCCESS;
    eai_ports_info_t ports_info;
    eai_tensor_size_info_t tensor_size_info;
    ports_info.input_or_output = input_or_output;
    ports_info.size = 0;
    *count = 0;

    eai_util_intf_t *p_util_intf = p_eai_data->p_eai_util_intf;
    eai_result = p_util_intf->eai_util_get_property(p_util_intf->eaiuh,
        model_id, EAI_PROP_PORTS_NUM, &ports_info);
    if (eai_result != EAI_SUCCESS) {
        MEAI_DBG(p_eai_data, DBG_ERROR_PRIO, "Fail to get eai property
            EAI_PROP_PORTS_NUM(%d)", EAI_PROP_PORTS_NUM);
        return MODULE_EAI_RET_FAIL;
    }
}

```

```

    if (ports_info.size > MAX_MODEL_INOUT_COUNT) {
        MEAI_DBG(p_eai_data, DBG_ERROR_PRIO, "Invalid port num
            received %u(Max allowed %u)", ports_info.size,
            MAX_MODEL_INOUT_COUNT);
        return MODULE_EAI_RET_FAIL;
    }
    MEAI_DBG(p_eai_data, DBG_HIGH_PRIO, "%s port count is %d",
        input_or_output?"Output":"Input", ports_info.size);
    *count = ports_info.size;
    for (uint32_t i = 0; i < *count; i++) {
        tensor_size_info.input_or_output = input_or_output;
        tensor_size_info.index = (uint32_t)i;
        tensor_size_info.size = 0;
        eai_result = p_util_intf->eai_util_get_property(
            p_util_intf->eaiuh, model_id,
            EAI_PROP_TENSOR_SIZE_INFO, &tensor_size_info);
        if (eai_result != EAI_SUCCESS){
            MEAI_DBG(p_eai_data, DBG_ERROR_PRIO, "Fail to get eai
                property EAI_PROP_TENSOR_SIZE_INFO(%d)",
                EAI_PROP_TENSOR_SIZE_INFO);
            return MODULE_EAI_RET_FAIL;
        }
        sizes[i] = tensor_size_info.size;
        MEAI_DBG(p_eai_data, DBG_HIGH_PRIO, "%s Tensor %d, size = %d",
            input_or_output?"Output":"Input", i,
            tensor_size_info.size);
    }
    return MODULE_EAI_RET_SUCCESS;
}

/* To release all input/output tensor related memory */
module_eai_result_t reset_input_output_info(eai_data_t *p_eai_data)
{
    EAI_RESULT eai_result = EAI_SUCCESS;
    eai_util_intf_t *p_util_intf;
    module_eai_result_t result = MODULE_EAI_RET_SUCCESS;
    eai_register_buffer_t buf_info = { NULL, 0,
        EAI_REG_BUF_FLAGS_DEREGISTER };
    eai_register_buffer_info_t buf_prop = { 1, &buf_info };
    p_util_intf = p_eai_data->p_eai_util_intf;

```

```

p_eai_data->input_count = 0;
memset(p_eai_data->input_size, 0x00, sizeof(p_eai_data->input_size));
memset(p_eai_data->input_type, 0x00, sizeof(p_eai_data->input_type));
memset(p_eai_data->inputs_buffer_info, 0x00,
        sizeof(p_eai_data->inputs_buffer_info));
p_eai_data->output_count = 0;
memset(p_eai_data->output_size, 0x00,
        sizeof(p_eai_data->output_size));
memset(p_eai_data->output_type, 0x00,
        sizeof(p_eai_data->output_type));
memset(p_eai_data->outputs_buffer_info, 0x00,
        sizeof(p_eai_data->outputs_buffer_info));
memset(&(p_eai_data->batch_info), 0x00, sizeof(eai_batch_info_t));

if (FALSE == p_eai_data->reuse_scratch) {
    for (int i = 0; i < p_eai_data->input_count; i++) {
        if (p_eai_data->input_addr[i]) {
            buf_info.buf = p_eai_data->input_addr[i];
            buf_info.buf_size = p_eai_data->input_size[i];

            if (0 == p_eai_data->enable_io_buf_reg) {
                eai_result = p_util_intf->eai_util_set_property(
                    p_util_intf->eaiuh,
                    p_eai_data->model_id,
                    EAI_PROP_REGISTER_BUFFER,
                    &buf_prop);

                if ((EAI_SUCCESS != eai_result) &&
                    (EAI_UNSUPPORTED_PROP != eai_result)) {
                    MEAI_DBG(p_eai_data, DBG_ERROR_PRIO,
                        "Fail to deregister io buffer, error %d",
                        eai_result);
                    result = MODULE_EAI_RET_FAIL;
                    goto bail;
                }
            }

            posal_memory_aligned_free(p_eai_data->input_addr[i]);
            p_eai_data->input_addr[i] = NULL;
        }
    }
}

```

```

    }
    for (int i = 0; i < p_eai_data->output_count; i++) {
        if (p_eai_data->output_addr[i]) {
            buf_info.buf = p_eai_data->output_addr[i];
            buf_info.buf_size = p_eai_data->output_size[i];
            if (0 == p_eai_data->enable_io_buf_reg) {
                eai_result = p_util_intf->eai_util_set_property(
                    p_util_intf->eaiuh,
                    p_eai_data->model_id,
                    EAI_PROP_REGISTER_BUFFER,
                    &buf_prop);

                if ((EAI_SUCCESS != eai_result) &&
                    (EAI_UNSUPPORTED_PROP != eai_result)) {
                    MEAI_DBG(p_eai_data, DBG_ERROR_PRIO,
                        "Fail to deregister io buffer, error %d",
                        eai_result);
                    result = MODULE_EAI_RET_FAIL;
                    goto bail;
                }
            }
            posal_memory_aligned_free(p_eai_data->output_addr[i]);
            p_eai_data->output_addr[i] = NULL;
        }
    }
} else {
    memset(p_eai_data->input_addr, 0x00,
        sizeof(p_eai_data->input_addr));
    memset(p_eai_data->output_addr, 0x00,
        sizeof(p_eai_data->output_addr));
}
bail:
    return result;
}

```

5.2.3 Execute

The function, `module_eai_execute()`, explains the execute call flow. This function does the following:

- Checks the model state, if it is in ready state (MODULE_EAI_STATE_READY) then it continues otherwise, it returns MODULE_EAI_RET_BAD_STATE
- Copies input data from the input pointer to the p_eai_data input pointer
- Set configure as TRUE to prevent low power modes for ML HW use case.
- Calls eai_util_execute() for processing
- Set configure as False

This function should be called repeatedly from the CAPI module's process function call.

```

module_eai_result_t module_eai_execute(eai_data_t *p_eai_data,
uint8_t *input, uint32_t input_len, uint8_t *output, uint32_t output_len)
{
    EAI_RESULT = EAI_SUCCESS;
    EAI_RESULT eai_result1 = EAI_SUCCESS;
    module_eai_result_t result = MODULE_EAI_RET_SUCCESS;
    eai_util_intf_t *p_util_intf = p_eai_data->p_eai_util_intf;

    if (MODULE_EAI_STATE_READY != p_eai_data->state) {
        MEAI_DBG(p_eai_data, DBG_ERROR_PRIO, "Cur state = %d, expected
            MODULE_EAI_STATE_READY", p_eai_data->state);
        return MODULE_EAI_RET_BAD_STATE;
    }

    if (input_len < p_eai_data->inputs_buffer_info[0].buffer_size) {
        MEAI_DBG_ISLAND(p_eai_data, DBG_HIGH_PRIO, "Input size not enough,
            skipping eai_util_execute");
        return result;
    }

    /* TODO: Extract features from input and fill input tensor */
    /* Currently calling eai_execute() with raw pcm sample data */
    memcpy(p_eai_data->inputs_buffer_info[0].addr, input,
        p_eai_data->inputs_buffer_info[0].buffer_size);
    MEAI_DBG_ISLAND(p_eai_data, DBG_HIGH_PRIO, "Calling eai_util_execute
        for eaiuh = %p, model_id = %d", p_util_intf->eaiuh,
        p_eai_data->model_id);

    /* Called to prevent low power modes for ML Hw use case */
    eai_util_mla_configure_low_power_modes_t mla_lpm_config;
    mla_lpm_config.configure = TRUE;
    eai_result = p_util_intf->eai_util_set_property( p_util_intf->eaiuh,
        0, EAI_UTIL_PROP_MLA_CONFIGURE_LOW_POWER_MODES,

```



```

        (void *) (&mmla_lpm_config));
    if (eai_result != EAI_SUCCESS) {
        MEAI_DBG(p_eai_data, DBG_ERROR_PRIO,
            "EAI_UTIL_PROP_MLA_CONFIGURE_LOW_POWER_MODES Set failed,"
            " result:0x%x", eai_result);
        result = MODULE_EAI_RET_FAIL;
    }
    eai_result = p_util_intf->eai_util_execute(p_util_intf->eaiuh,
        p_eai_data->model_id, &(p_eai_data->batch_info), 1);
    mmla_lpm_config.configure = FALSE;
    eai_result1 = p_util_intf->eai_util_set_property( p_util_intf->eaiuh,
        0, EAI_UTIL_PROP_MLA_CONFIGURE_LOW_POWER_MODES,
        (void *) (&mmla_lpm_config));
    if (eai_result1 != EAI_SUCCESS) {
        MEAI_DBG(p_eai_data, DBG_ERROR_PRIO,
            "EAI_UTIL_PROP_MLA_CONFIGURE_LOW_POWER_MODES Set failed,"
            " result:0x%x", eai_result1);
        result = MODULE_EAI_RET_FAIL;
    }
    if ((EAI_SUCCESS != eai_result) && (EAI_NEED_MORE != eai_result) &&
        (EAI_HAS_MORE != eai_result)) {
        MEAI_DBG_ISLAND(((int)p_eai_data), DBG_ERROR_PRIO,
            "eai_util_execute failed, error %d", eai_result);
        return MODULE_EAI_RET_FAIL;
    }
    /* TODO: Copy the output data/Take action based on output tensor */
    return result;
}

```

5.2.4 Deinitialization

The functions `module_eai_unload_and_release_model()` and `module_eai_deinit()` explain the deinitialization call flow.

- `module_eai_unload_and_release_model()` - Unloads and releases the loaded eAI model. It checks the state of the model and does the following:
 - If model state is neither in `EAI_STATE_READY`, `EAI_STATE_MODEL_LOADED`, or `EAI_STATE_MODEL_RVCD`, it returns `EAI_RET_BAD_STATE`
 - If model state is either `EAI_STATE_READY` or `EAI_STATE_MODEL_LOADED`, it unloads the model via `module_eai_unload_model()` function call.

- If model state is `EAI_STATE_MODEL_RVCD`, it releases the model via `module_eai_release_model()` function call.
- This function can be called from CAPI module's end callback function. After calling this function, the eAI Utility must be destroyed by calling the `module_eai_deinit()` function.
- `module_eai_deinit()` – Destroys the instances of `eai_util`. It does the following:
 - Destroys eAI Utility instances by calling the `eai_util_destroy()`
 - On success, it updates the model state to `MODULE_EAI_STATE_NULL`
 - This function can be called from the CAPI module's end callback function only after the eAI model is unloaded and released successfully by calling `module_eai_unload_and_release_model()`

The following helper functions are used to deinitialize the eAI Utility instance:

- `module_eai_unload_model()` – Unload the loaded eAI model. It does the following:
 - Checks the model state. If it is neither `MODULE_EAI_STATE_MODEL_LOADED` or `MODULE_EAI_STATE_READY`, it returns `MODULE_EAI_RET_BAD_STATE`
 - If the model state is `MODULE_EAI_STATE_READY`, it transits the current model state to loaded by resetting all input/output tensor information and releasing the scratch memory (if any)
 - Calls `eai_util_deinit()` to deinitialize the eAI Utility instance and updates the model state to `MODULE_EAI_STATE_MODEL_RVCD`
- `module_eai_release_model()` – Releases the unloaded eAI model. It does the following:
 - Checks the model state. If it is not in `MODULE_EAI_STATE_MODEL_RVCD`, then it returns `MODULE_EAI_RET_BAD_STATE`.
 - If `is_model_persistent` is `TRUE`, then it releases the model memory and updates the model state to `MODULE_EAI_STATE_INIT`

/* To unload and release the loaded eAI model */

```
module_eai_result_t module_eai_unload_and_release_model(eai_data_t
*p_eai_data)
{
    module_eai_result_t result = MODULE_EAI_RET_SUCCESS;
    if (!p_eai_data) {
        MEAI_DBG(p_eai_data, DBG_ERROR_PRIO, "Invalid input arguments");
        return MODULE_EAI_RET_BAD_PARAM;
    }
    if ((MODULE_EAI_STATE_MODEL_LOADED != p_eai_data->state) &&
        (MODULE_EAI_STATE_READY != p_eai_data->state) &&
        (MODULE_EAI_STATE_MODEL_RVCD != p_eai_data->state)) {
        MEAI_DBG(p_eai_data, DBG_ERROR_PRIO, "Cur state = %d, expected "
```

```

        "MODULE_EAI_STATE_MODEL_LOADED/READY/MODEL_RCVD",
        p_eai_data->state);
    return MODULE_EAI_RET_BAD_STATE;
}

if ((MODULE_EAI_STATE_READY == p_eai_data->state) ||
    (MODULE_EAI_STATE_MODEL_LOADED == p_eai_data->state)) {
    result = module_eai_unload_model(p_eai_data);
    if (MODULE_EAI_RET_SUCCESS != result) {
        MEAI_DBG(p_eai_data, DBG_ERROR_PRIO, "Fail to unload model, err
            = %d", result);
        return MODULE_EAI_RET_FAIL;
    }
}

if (MODULE_EAI_STATE_MODEL_RVCD == p_eai_data->state) {
    result = module_eai_release_model(p_eai_data);
    if (MODULE_EAI_RET_SUCCESS != result) {
        MEAI_DBG(p_eai_data, DBG_ERROR_PRIO, "Fail to release model,
            err = %d", result);
        return MODULE_EAI_RET_FAIL;
    }
}

return result;
}

/* To deinitialize the eAI util instance */
module_eai_result_t module_eai_deinit(eai_data_t *p_eai_data)
{
    module_eai_result_t result = MODULE_EAI_RET_SUCCESS;
    if (!p_eai_data) {
        MEAI_DBG(p_eai_data, DBG_ERROR_PRIO, "Invalid input arguments");
        return MODULE_EAI_RET_BAD_PARAM;
    }
    MEAI_DBG(p_eai_data, DBG_HIGH_PRIO, "In %s...", __func__);
    if (MODULE_EAI_STATE_INIT != p_eai_data->state) {
        MEAI_DBG(p_eai_data, DBG_ERROR_PRIO, "Cur state = %d, expected
            MODULE_EAI_STATE_INIT", p_eai_data->state);
        return MODULE_EAI_RET_BAD_STATE;
    }
    /* destroy eai_util */

```

```

    if (p_eai_data->p_eai_util_intf) {
        eai_util_destroy(p_eai_data->p_eai_util_intf);
        p_eai_data->p_eai_util_intf = NULL;
    }
    if (p_eai_data) {
        memset(p_eai_data, 0x00, sizeof(eai_data_t));
    }
    /* Update state to NULL */
    p_eai_data->state = MODULE_EAI_STATE_NULL;
    return result;
}

/* Helper functions */
/* Function to unload the eAI model*/
module_eai_result_t module_eai_unload_model(eai_data_t *p_eai_data)
{
    EAI_RESULT = EAI_SUCCESS;
    module_eai_result_t result = MODULE_EAI_RET_SUCCESS;
    if (!p_eai_data) {
        MEAI_DBG(p_eai_data, DBG_ERROR_PRIO, "Invalid input arguments");
        return MODULE_EAI_RET_BAD_PARAM;
    }
    eai_util_intf_t *p_util_intf = p_eai_data->p_eai_util_intf;
    if ((MODULE_EAI_STATE_MODEL_LOADED != p_eai_data->state) &&
        (MODULE_EAI_STATE_READY != p_eai_data->state)) {
        MEAI_DBG(p_eai_data, DBG_ERROR_PRIO, "Cur state = %d, expected
        MODULE_EAI_STATE_MODEL_LOADED/READY", p_eai_data->state);
        return MODULE_EAI_RET_BAD_STATE;
    }
    /* If current state is ready, transition to loaded first */
    if (MODULE_EAI_STATE_READY == p_eai_data->state) {
        if (p_eai_data->scratch_mem) {
            posal_memory_free(p_eai_data->scratch_mem);
            p_eai_data->scratch_mem = NULL;
            p_eai_data->scratch_mem_size = 0;
        }
        result = reset_input_output_info(p_eai_data);
        if (MODULE_EAI_RET_SUCCESS != result) {
            MEAI_DBG(p_eai_data, DBG_ERROR_PRIO, "Fail to reset input

```

```

        output buffers, %d", result);
        return MODULE_EAI_RET_FAIL;
    }
    p_eai_data->state = MODULE_EAI_STATE_MODEL_LOADED;
}
/* unload the eai model now */
if (p_eai_data->model != NULL) {
    eai_result = p_util_intf->eai_util_deinit(
        p_util_intf->eaiuh, p_eai_data->model_id);
    if (EAI_SUCCESS != eai_result) {
        /* check if we can do if deinit fails,
         * currently ignoring this failure */
        MEAI_DBG(p_eai_data, DBG_ERROR_PRIO, "Fail to deinit eAI
            instance. This will be ignored");
    }
}
p_eai_data->state = MODULE_EAI_STATE_MODEL_RVCD;
return result;
}

/* Function to release the eAI model*/
module_eai_result_t module_eai_release_model(eai_data_t *p_eai_data)
{
    if (!p_eai_data) {
        MEAI_DBG(p_eai_data, DBG_ERROR_PRIO, "Invalid input arguments");
        return MODULE_EAI_RET_BAD_PARAM;
    }
    if (MODULE_EAI_STATE_MODEL_RVCD != p_eai_data->state) {
        MEAI_DBG(p_eai_data, DBG_ERROR_PRIO, "Cur state = %d, expected
            MODULE_EAI_STATE_MODEL_RVCD", p_eai_data->state);
        return MODULE_EAI_RET_BAD_STATE;
    }
    /* Allocate model memory and copy the model */
    if (p_eai_data->model) {
        if (!(p_eai_data->is_model_persistent)) {
            /* Release the memory only if we allocated for it */
            posal_memory_free(p_eai_data->model);
        }
        p_eai_data->model = NULL;
    }
}

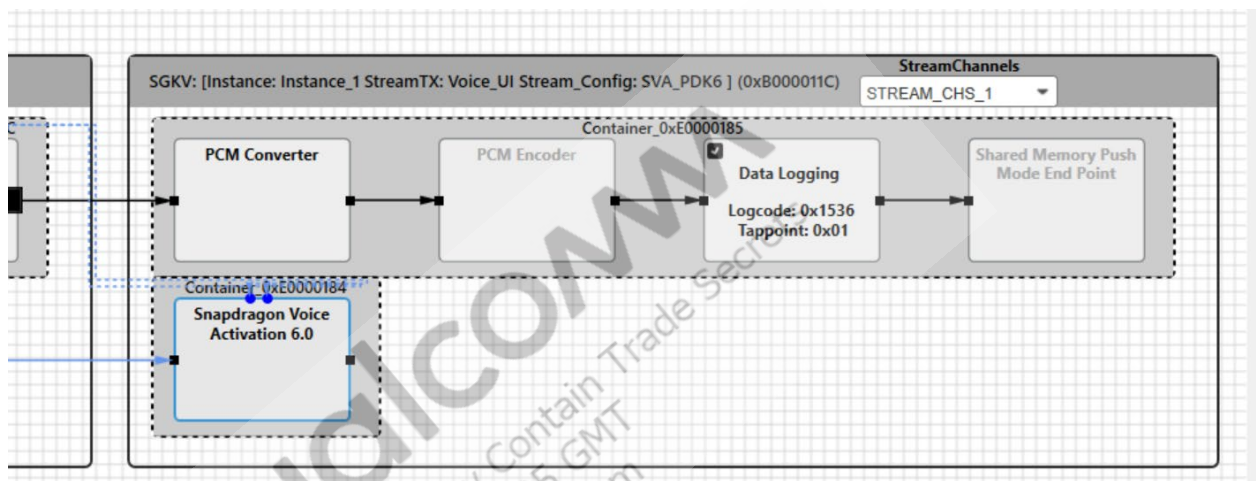
```

```
p_eai_data->model_size = 0;
p_eai_data->model_id = -1;
/* Change the state to init */
p_eai_data->state = MODULE_EAI_STATE_INIT;
return MODULE_EAI_RET_SUCCESS;
}
```

Qualcomm
Confidential - May Contain Trade Secrets
2026-05-12 06:36:25 GMT
hyungchul.won@lge.com

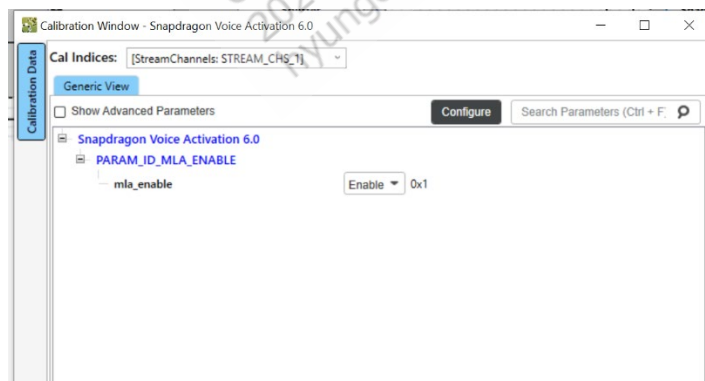
6 Power measurements with eNPU enabled/disabled

In QTI release builds, by default, the SVA6.0 module uses the eNPU hardware for the Voice Activation use case. In order to measure the power numbers with the eNPU enabled/disabled, open the SVA use case graph from ACDB and double click on the SVA6.0 module view its parameters.



6.1 Power measurement with eNPU hardware enabled

By default, having `mla_enable` set to enabled indicates the SVA module will use the eNPU hardware.

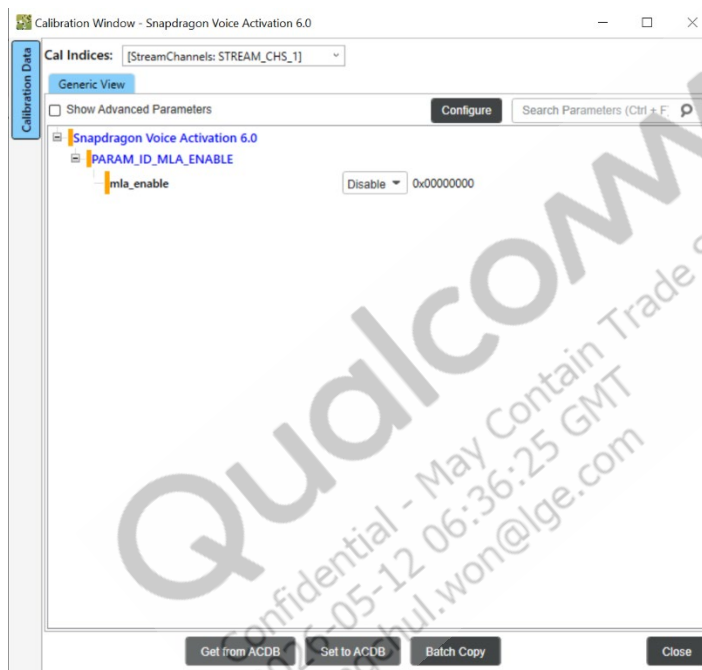


To measure the power numbers with eNPU hardware enabled for the SVA6.0 module for the Voice UI use case:

1. Push the QTI default ACDB graph and make the necessary changes to match the OEM's hardware configuration such as the input mic source devices.
2. Put the device in Airplane mode (Wi-Fi, Bluetooth, GPS, and sensors off) and measure the power once stable for 30 sec.
3. Start the Voice Activation use case and measure the power once stable for 30 sec.
4. Take the delta.

6.2 Power measurement with eNPU hardware disabled

Set `mfa_enable` to disabled to cause the SVA module to not perform eNPU offloading.



To measure the power numbers with eNPU hardware disabled for the SVA6.0 module for the Voice UI use case:

1. Push the QTI default ACDB graph and make the necessary changes to match the OEM's hardware configuration such as the input mic source devices.
2. Open the SVA6.0 module and change the `mfa_enable` calibration key to Disabled.
3. Click **Set to ACDB**.
4. Put the device in Airplane mode (Wi-Fi, Bluetooth, GPS, and sensors off) and measure the power once stable for 30 sec.
5. Start the Voice Activation use case and measure the power once stable for 30 sec.
6. Take the delta.

A References

A.1 Related documents

Title	Number
Qualcomm Technologies, Inc.	
<i>eAI Framework Architecture Overview</i>	80-PK177-55
<i>Low Power Island Voice Activation in AudioReach</i>	80-VN500-20
<i>CAPI Custom Module Integration into AudioReach Signal Processing Framework</i>	80-VN500-17

A.2 Acronyms and terms

Acronym or term	Definition
eAI	Embedded Artificial Intelligence
CAPI	Common Audio Processor Interface
LPASS	Low Power Audio Subsystem
QDSP/QDSP6	Qualcomm Hexagon Digital Signal Processor
eNPU/MLA	Embedded AI accelerator within LPASS also referred as Machine Learning Accelerator
ADSP	Audio Digital Signal Processor
LPI	Low Power Island
TCM	Tightly Coupled Memory
DDR	External Memory